

Descripción

La vista estática es la base de UML. Los elementos de la vista estática de un modelo son los conceptos significativos en una aplicación, incluyendo conceptos del mundo real, conceptos abstractos, conceptos de implementación, conceptos de computación y todo tipo de conceptos encontrados en los sistemas. Por ejemplo, un sistema de entradas para un teatro tiene conceptos tales como entradas, reservas, planes de suscripción, algoritmos de la asignación de asientos, páginas Web interactivas para hacer pedidos, y datos de archivo para redundancia.

La vista estática captura la estructura del objeto. Un sistema orientado a objetos unifica la estructura de datos y características del comportamiento en una sola estructura de objeto. La vista estática incluye todo lo concerniente a las estructuras de datos tradicionales, así como la organización de las operaciones sobre los datos. Los datos y las operaciones son cuantificadas en clases. En la perspectiva orientada a objetos, los datos y el comportamiento se relacionan estrechamente. Por ejemplo, un objeto **de entrada** lleva datos, como su precio, fecha de la representación, y número de asiento, así como operaciones sobre él, como reservar o calcular su precio con un descuento especial.

La vista estática describe entidades de comportamiento como elementos de modelado discretos, pero no contiene los detalles de su comportamiento dinámico. Los trata como elementos para ser nombrados, poseídos por las clases, e invocados. Su ejecución dinámica es descrita por otras vistas que muestran los detalles internos de su faceta dinámica. Estas otras vistas incluyen la vista de interacción y la vista de máquina de estados. Las vistas dinámicas requieren la vista estática para describir las cosas que interactúan dinámicamente: no se puede decir *cómo* interactúa algo sin decir primero *qué* está interactuando. La vista estática es la base sobre la que se construyen las otras vistas.

Los elementos clave en la vista estática son los clasificadores y sus relaciones. Un clasificador es un elemento de modelado que describe cosas. Hay varias clases de clasificadores, como las clases, interfaces, y tipos de datos. Los aspectos de comportamiento son materializados por otros clasificadores, como los casos de uso y las señales. Los propósitos de implementación están detrás de varias clases de clasificadores, tales como subsistemas, componentes, y nodos.

Los modelos grandes se deben organizar en unidades más pequeñas para la comprensión humana y la reutilización. Un paquete es una unidad de organización de uso general para poseer y manejar el contenido de un modelo. Cada elemento está contenido en algún paquete. Un modelo es un paquete que describe una vista completa de un sistema y se puede utilizar más o menos independientemente de otros modelos; es la raíz poseedora de los paquetes más detallados que describen el sistema.

Un objeto es una unidad discreta, fuera de la cual el modelador entiende y construye un sistema. Es una instancia de una clase, es decir, un individuo con identidad cuya estructura y comportamiento son descritos por la clase. Un objeto es una pieza de estado identificable con un comportamiento bien definido que puede ser invocado.

Las relaciones entre clasificadores son asociación, generalización, y varias clases de dependencia, como la realización y el uso.

Clasificadores

Un clasificador es un concepto discreto en el modelo, que tiene identidad, estado, comportamiento, y relaciones. Las clases de clasificadores incluyen la clase, la interfaz, y los tipos de datos. Otras clases de clasificadores son materializaciones de conceptos de comportamiento, de cosas del entorno, o de estructuras de implementación. Estos clasificadores incluyen caso de uso, actor, componente, nodo, y subsistema. La Tabla 4.1 enumera las distintas clases de clasificadores y sus funciones. El término clasificador del metamodelo incluye todos estos conceptos, pero como la clase es el término más familiar, lo discutiremos primero y definiremos los otros conceptos por diferencia con ella.

Clase. Una clase representa un concepto discreto dentro de la aplicación que se está modelando: una cosa física (tal como un aeroplano), una cosa de negocios (tal como un pedido), una cosa lógica (tal como un horario de difusión), una cosa de una aplicación (tal como un botón de cancelar), una cosa del computador (tal como una tabla hash), o una cosa del comportamiento (tal como una tarea). Una clase es el descriptor de un conjunto de objetos con una estructura, comportamiento, y relaciones similares. Todos los atributos y operaciones están unidos a clases o a otros clasificadores. Las clases son los focos alrededor de los cuales se organizan los sistemas orientados a objetos.

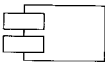
Un objeto es una entidad discreta con identidad, estado y un comportamiento invocable. Los objetos son piezas individuales, fuera de las cuales se construye un sistema ejecutable; las clases son los conceptos individuales, por los cuales se entiende y describe la multiplicidad de objetos individuales.

Una clase define un conjunto de objetos que tienen estado y comportamiento. El estado es descrito por atributos y asociaciones. Los atributos se utilizan generalmente para los valores puros de los datos sin identidad, tales como números y cadenas, y las asociaciones se utilizan para

Suscripción	nombre de clase
serie: Cadena categoriaDePrecio: Categoría número: Entero	atributos
conocerCoste (): Dinero reservar (serie: Cadena, nivel: NivelDe Asiento) cancelar ()	operaciones

Figura 4.1 Notación de clases

Tabla 4.1 Tipos de Clasificadores

<i>Clasificador</i>	<i>Función</i>	<i>Notación</i>
actor	Un usuario externo al sistema	
clase	Un concepto del sistema modelado	Nombre
clase en un estado	Una clase restringida a estar en el estado dado	Nombre[S]
rol	Clasificador restringido a un uso particular en una colaboración	rol:Nombre
componente	Una pieza física de un sistema	
tipo de dato	Un descriptor de un conjunto de valores primitivos que carecen de identidad	Nombre
interfaz	Un conjunto de operaciones con nombre que caracteriza un comportamiento	○ lnombre
nodo	Un recurso computacional	
señal	Una comunicación asíncrona entre objetos	«signal»
subsistema	Un paquete que es tratado como una unidad con una especificación, implementación, e identidad	«subsystem»
caso de uso	Una especificación del comportamiento de una identidad y su interacción con los agentes externos	

las conexiones entre objetos con identidad. Las piezas individuales de comportamiento invocable se describen mediante operaciones; un método es la implementación de una operación. La historia del curso de vida de un objeto es descrita por una máquina de estados, unida a una clase. La notación para una clase es un rectángulo con compartimentos para el nombre de la clase, los atributos, y las operaciones, según se muestra en la Figura 4.1.

Un conjunto de clases puede utilizar la relación de generalización y el mecanismo de herencia construidos en ella para compartir piezas comunes de estado y descripción del comportamiento. La generalización relaciona clases más específicas (subclases) con clases más ge-

nerales (superclases) que contienen propiedades comunes a varias subclases. Una clase puede tener cero o más padres (superclases) y cero o más hijos (subclases). Una clase hereda estado y descripción del comportamiento de sus padres y de otros antecesores, y define el estado y descripción del comportamiento que sus hijos y otros descendientes heredan.

Una clase tiene un nombre único dentro de su contenedor, que es generalmente un paquete, pero algunas veces es otra clase. La clase tiene visibilidad con respecto a su contenedor; la visibilidad especifica cómo puede ser utilizada por otras clases externas al contenedor. Una clase tiene una multiplicidad que especifica cuantas instancias de ella pueden existir. La mayoría de las veces, es muchos (cero o más, sin límite explícito), pero existen clases unitarias de las que existe una sola instancia durante la ejecución.

Interfaz. Una interfaz es la descripción del comportamiento de objetos sin dar su implementación o estado; una interfaz contiene operaciones pero no atributos, y no tiene asociaciones salientes que muestren la visibilidad desde la propia interfaz.

Una o más clases o componentes pueden realizar una interfaz, y cada clase implementa las operaciones de la interfaz.

Tipo de datos. Un tipo de datos es la descripción de los valores primitivos, que carecen de identidad (existencia independiente y posibilidad de efectos secundarios). Los tipos de datos incluyen números, cadenas, y valores enumerados. Los tipos de datos son pasados por valor y son entidades inmutables. Un tipo de dato no tiene atributos pero puede tener operaciones. Las operaciones no modifican valores de los datos, sino que pueden devolver valores de datos como resultados.

Niveles de significado. Las clases pueden existir en varios niveles de significación en un modelo, incluyendo los niveles de análisis, diseño, e implementación. Al representar conceptos del mundo real, es importante capturar el estado, las relaciones, y el comportamiento del mundo real. Pero los conceptos de implementación, tales como ocultación de información, eficacia, visibilidad, y métodos, no son conceptos relevantes del mundo real (son conceptos relevantes de diseño). Muchas propiedades potenciales de una clase son simplemente inaplicables en este nivel. Una clase en el análisis representa un concepto lógico en el dominio de la aplicación o en la aplicación misma. El modelo de análisis debe ser una representación mínima del sistema que se está modelando, suficiente para capturar la lógica esencial del sistema, sin entrar en temas de rendimiento o construcción.

Al representar un diseño de alto nivel, conceptos tales como hallar los estados de una clase particular, la eficacia de la navegación entre objetos, la separación del comportamiento externo y de la implementación interna, y la especificación de las operaciones exactas son relevantes a una clase. Una clase en el diseño representa la decisión de empaquetar la información de estado y las operaciones en una unidad discreta. Captura las decisiones clave del diseño, la localización de la información y la funcionalidad de los objetos. Las clases en el diseño contienen aspectos del mundo real y aspectos del sistema informático.

Finalmente, al representar código del lenguaje de programación, la forma de una clase se asemeja mucho a la del lenguaje de programación elegido, y se puede renunciar a algunas capacidades de una clase general, si no tienen ninguna implementación directa en el lenguaje. Una clase para implementación se corresponde directamente con el código del lenguaje programación.

El mismo sistema puede contener más de un nivel de clase; las clases orientadas a la implementación pueden realizar clases más lógicas en el modelo. Una clase de implementación representa la declaración de una clase como la encontramos en un lenguaje de programación particular. Captura la forma exacta de una clase, según lo requerido por el lenguaje. En muchos casos, sin embargo, el análisis, el diseño, y la información de la implementación se pueden anidar en una sola clase.

Relaciones

Las relaciones entre clasificadores son asociación, generalización, flujo, y varias clases de dependencia, que incluyen la realización y el uso (véase la Tabla 4.2).

La relación de asociación describe conexiones semánticas entre los objetos individuales de clases dadas. Las asociaciones proporcionan las conexiones, con las cuales los objetos de diversas clases pueden interactuar. Las relaciones restantes relacionan las descripciones de clasificadores con ellos mismos, y no con sus instancias.

La relación de generalización relaciona descripciones generales de los clasificadores padre (superclases) con clasificadores hijos especializados (subclases). La generalización facilita la descripción de clasificadores, sin piezas de declaración incremental, cada uno de los cuales se agrega a la descripción heredada de sus antecesores. El mecanismo de herencia construye descripciones completas de clasificadores a partir de descripciones incrementales que utilizan relaciones de generalización. La generalización y la herencia permiten a diferentes clasificadores, compartir atributos, operaciones, y relaciones que tienen en común, sin repetirlas.

La relación de realización relaciona una especificación con una implementación. Una interfaz es una especificación del comportamiento sin la implementación; una clase incluye la estructura de implementación. Una o más clases pueden realizar una interfaz, y cada clase implementa las operaciones de interfaz.

Tabla 4.2 Tipos de Relaciones

<i>Relación</i>	<i>Función</i>	<i>Notación</i>
asociación	Una descripción de una conexión entre instancias de clases	— — —
dependencia	Una relación entre dos elementos del modelo	— — — ➤
flujo	Una relación entre dos versiones de un objeto en sucesivas veces	— — — ➤
generalización	Una relación entre una descripción más general y una variedad más específica de la general, usada para herencia	— ➤
realización	Relación entre una especificación y su implementación	— — ➤
uso	Una situación en la que un elemento requiere otro para su correcto funcionamiento	— — — ➤

La relación de flujo relaciona dos versiones de un objeto en momentos sucesivos. Representa una transformación del valor, estado, o localización de un objeto. La relación de flujo puede conectar roles en una interacción. Las variedades de flujo son conversión (dos versiones del mismo objeto) y copia (un nuevo objeto creado de un objeto existente).

La relación de dependencia relaciona las clases cuyo comportamiento o implementación afecta a otras clases. Hay varias clases de dependencia además de la realización, incluyendo la traza (una conexión leve entre elementos de diversos modelos), refinamiento (la correspondencia entre dos niveles de significación), uso (un requisito para la presencia de otro elemento dentro de un solo modelo), y ligadura (la asignación de valores a los parámetros de una plantilla). La dependencia de uso se utiliza con frecuencia para representar relaciones de implementación, tales como relaciones al nivel de código. La dependencia es particularmente útil cuando está resumida en unidades de organización del modelo, tales como paquetes, en los cuales se muestra la estructura arquitectónica de un sistema. Por ejemplo, las restricciones para la compilación se pueden mostrar con dependencias.

Asociaciones

Una asociación describe conexiones discretas entre objetos u otras instancias de un sistema. Una asociación relaciona una lista ordenada (tupla) de dos o más clasificadores, con las repeticiones permitidas. El tipo más común de asociación es una asociación binaria entre un par de clasificadores. Una instancia de una asociación es un enlace. Un enlace abarca una tupla (una lista ordenada) de objetos, cada uno dibujado a partir de su clase correspondiente. Un enlace binario abarca un par de objetos.

Las asociaciones llevan la información sobre relaciones entre objetos en un sistema. Cuando se ejecuta un sistema, los enlaces entre objetos se crean y se destruyen. Las asociaciones son el “pegamento” que mantiene unido un sistema. Sin asociaciones, no hay nada más que clases aisladas que no trabajan juntas.

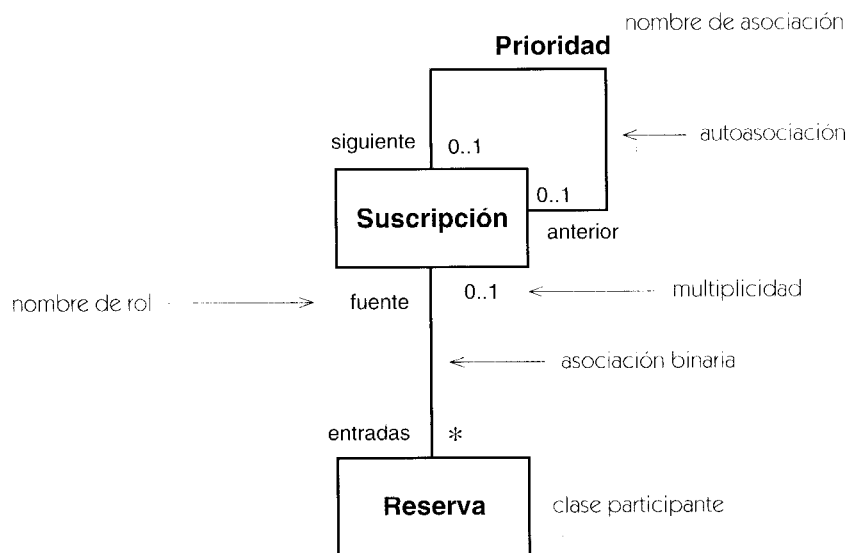


Figura 4.2 Notación de asociación

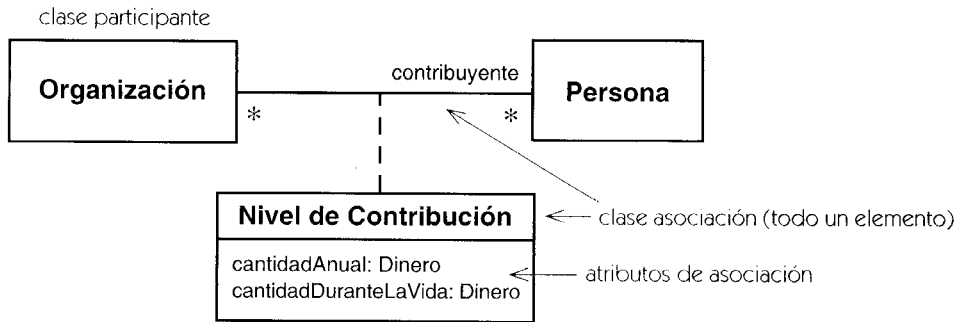


Figura 4.3 Clase de asociación

Un solo objeto se puede asociar a sí mismo si la misma clase aparece más de una vez en una asociación. Si la misma clase aparece dos veces en una asociación, las dos instancias no tienen que ser el mismo objeto, y no lo son generalmente.

Cada conexión de una asociación a una clase se llama extremo de la asociación. La mayoría de la información sobre una asociación se une a uno de sus extremos. Los extremos de la asociación pueden tener nombres (nombres de rol) y visibilidad. La propiedad más importante que tienen es la multiplicidad: cuantas instancias de una clase se pueden relacionar con una instancia de otra clase. La multiplicidad es más útil para las asociaciones binarias porque su definición para las asociaciones de aridad- n es complicada.

La notación para una asociación binaria es una línea o una trayectoria que conecta las clases que participan. El nombre de asociación se pone a lo largo de la línea, con el nombre de rol y la multiplicidad en cada extremo, según lo mostrado en la Figura 4.2.

Una asociación puede también tener atributos por sí misma, en cuyo caso es una asociación y una clase, es decir, una clase asociación (véase la Figura 4.3). Si un atributo de la asociación es único dentro de un conjunto de objetos relacionados, entonces es un calificador (véase la Figura 4.4). Un calificador es un valor que selecciona un objeto único del conjunto de objetos relacionados a través de una asociación. Las tablas de valores y los vectores se pueden modelar como asociaciones calificadas. Los calificadores son importantes para modelar nombres y códigos de identificación. Los calificadores también modelan índices en un modelo de diseño.

Durante el análisis, las asociaciones representan relaciones lógicas entre objetos. No hay gran necesidad de imponer la dirección, o de tratar sobre cómo implementarla. Las asociaciones redundantes deben ser evitadas porque no añaden ninguna información lógica. Durante el diseño, las asociaciones capturan las decisiones de diseño sobre la estructura de datos, así como la se-

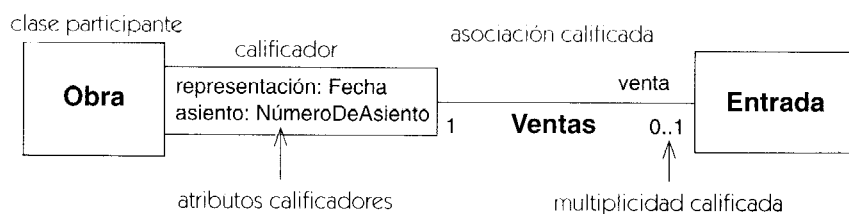


Figura 4.4 Asociación calificada

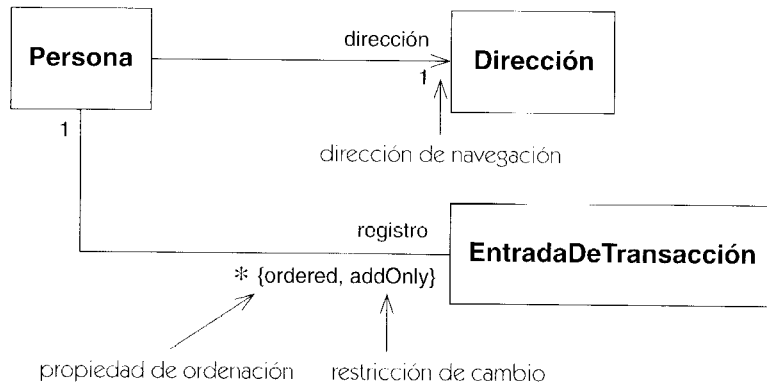


Figura 4.5 Propiedades del diseño de asociaciones

paración de responsabilidades entre clases. La direccionalidad de las asociaciones es importante, y las asociaciones redundantes pueden ser incluidas por eficacia en el acceso al objeto, así como para localizar la información en una clase particular. Sin embargo, en esta etapa de modelado, las asociaciones no se deben comparar con los punteros de C++. Una asociación navegable en la etapa de diseño representa información de estado disponible para una clase, pero puede implementarse en código del lenguaje de programación de varias maneras. La implementación puede ser un puntero, una clase contenedora embebida en una clase, o incluso una tabla de objetos totalmente separada. Otras clases de propiedades del diseño incluyen visibilidad y posibilidad de cambio de enlaces. La Figura 4.5 muestra algunas propiedades del diseño de asociaciones.

Agregación y composición. Una agregación es una asociación que representa una relación todo-parte. Se muestra adornando con un diamante hueco el extremo de la trayectoria unida a la clase agregada. Una composición es una forma más fuerte de asociación, en la cual el compuesto es el responsable único de gestionar sus partes, por ejemplo su asignación y desasignación. Se muestra con un diamante relleno adornando el extremo compuesto. Hay una asociación separada entre cada clase que representa una parte y la clase que representa el todo, pero por conveniencia, las trayectorias unidas al todo pueden ensamblarse juntas para dibujar el sistema entero de asociaciones como un árbol. La Figura 4.6 muestra un agregado y un compuesto.

Enlaces. Una instancia de una asociación es un enlace. Un enlace es una lista ordenada de referencias a objetos, cada uno de los cuales debe ser una instancia de la clase correspondiente en la asociación o una instancia de un descendiente de la clase. Los enlaces en un sistema constituyen parte del estado del sistema. Los enlaces no existen independientemente de los objetos; to-

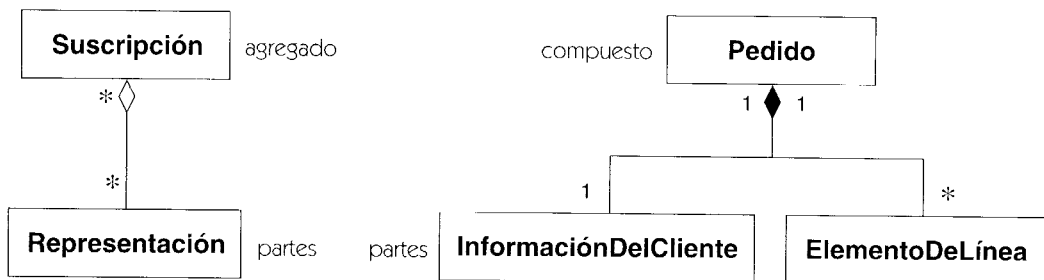


Figura 4.6 Agregación y Composición

man su identidad de los objetos que se relacionan (en términos de bases de datos, la lista de objetos es la clave para el enlace). Conceptualmente, una asociación es distinta de las clases que relaciona. En la práctica, las asociaciones se implementan a menudo usando punteros en las clases participantes, pero se pueden implementar como objetos contenedores, separados de las clases que conectan.

Bidireccionalidad. Los diferentes extremos de una asociación son distinguibles, incluso si dos de ellos implican la misma clase. Esto significa simplemente que diversos objetos de la misma clase pueden ser relacionados. Debido a que los extremos son distinguibles, una asociación no es simétrica (excepto en casos especiales); los extremos no pueden ser intercambiados. Esto es sólo sentido común: el sujeto y el predicado de un verbo no son permutables. A veces, se dice que una asociación es bidireccional. Esto significa que las relaciones lógicas trabajan en ambos sentidos. Esta declaración se entiende mal con frecuencia, incluso por algunos metodólogos. No significa que cada clase “conozca” a la otra clase, o que, en una implementación, sea posible tener acceso a cada clase desde la otra. Significa simplemente que cualquier relación lógica tiene un contrario, tanto si lo contrario es fácil de computar como si no. Las asociaciones se pueden marcar con direcciones de navegación, para asignar la posibilidad de atravesar una asociación en una dirección pero no en la otra, como una decisión del diseño.

¿Por qué es relacional el modelo básico, en vez de con punteros, que son un modelo más frecuente en lenguajes de programación? La razón es que un modelo procura capturar el objetivo subyacente a una implementación. Por consiguiente, si una relación entre dos clases se modela como un par de punteros, los punteros deben estar relacionados. El concepto de asociación reconoce que las relaciones son significativas en ambas direcciones, sin importar cómo se implementan. Es sencillo convertir una asociación en un par de punteros para su implementación, pero es muy difícil reconocer dos punteros que son contrarios entre sí, a menos que este hecho sea parte del modelo.

Generalización

La relación de generalización es una relación taxonómica entre una descripción más general y una descripción más específica, que se construye sobre ella y la extiende. La descripción más específica es completamente consistente con la más general (tiene todas sus propiedades, miembros y relaciones), y puede contener información adicional. Por ejemplo, una hipoteca es una clase más específica de préstamo. Una hipoteca guarda las propiedades básicas de un préstamo pero agrega propiedades adicionales, tales como una casa o como seguridad para el préstamo. La descripción más general se llama padre; un elemento en la clausura transitiva es un antecesor. La descripción más específica se llama hijo; un elemento en la clausura transitiva es un descendiente. En el ejemplo, el **préstamo** es la clase del padre y la **hipoteca** es la clase hija. La generalización se utiliza para los clasificadores (clases, interfaces, tipos de los datos, casos de uso, actores, señales, etcétera), paquetes, máquinas de estado, y otros elementos. Para las clases, los términos superclase y subclase se utilizan para el padre y el hijo.

Una generalización se dibuja como una flecha desde hijo al padre, con un triángulo hueco en el extremo conectado con el padre (Figura 4.7). Varias relaciones de generalización se pueden dibujar como un árbol con una punta de flecha que se ramifica en varias líneas a los hijos.

Propósito de la generalización. La generalización tiene dos propósitos. El primero debe definir las condiciones bajo las cuales una instancia de una clase (u otro elemento) puede ser

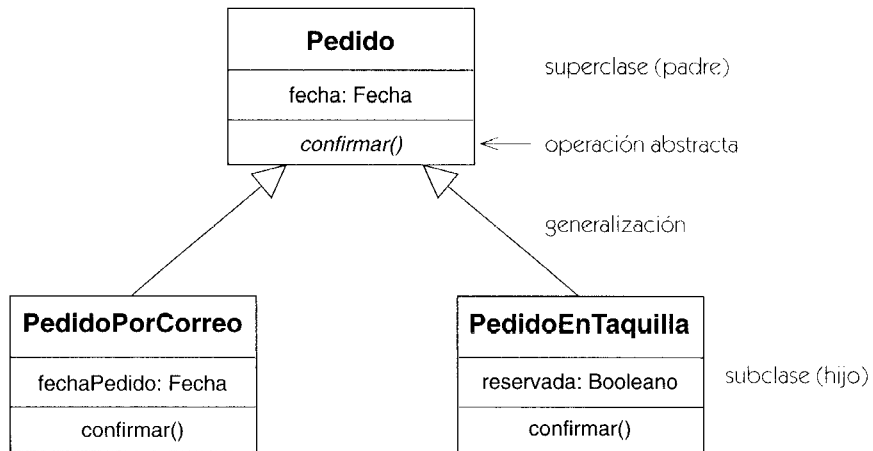


Figura 4.7 Notación de la generalización

utilizado cuando se declara una variable (tal como un parámetro o variable del procedimiento), conteniendo valores de una clase dada. Esto se llama principio de sustitución (de Bárbara Liskov). La regla es que una instancia de un descendiente se puede utilizar dondequiera que esté declarado el antecesor. Por ejemplo, si una variable se declara para contener préstamos, un objeto hipoteca es un valor legal para esa variable.

La generalización permite operaciones polimórficas, es decir, operaciones cuya implementación (método) es determinada por la clase de objeto a la que se aplican, en vez de ser indicada explícitamente por el llamador. Esto funciona porque una clase padre puede tener muchos hijos posibles, cada uno de los cuales implementa su propia variación de una operación, que se define a través de todo el conjunto de clases. Por ejemplo, el cálculo de intereses funcionaría de forma diferente en una hipoteca y en un préstamo para un automóvil, pero cada uno de ellos es una variación del cálculo de interés de la clase padre **Préstamo**. Cuando se declara una variable de la clase padre, cualquier objeto de clases hijas puede ser asociado a ella, lógicamente cada uno de ellos con sus propias operaciones según la clase a la que pertenezca. Esto es particularmente útil porque se pueden agregar nuevas clases más adelante, sin necesidad de modificar las llamadas polimórficas existentes. Por ejemplo, se podría agregar más adelante una nueva clase préstamo, y el código existente que utiliza la operación del **cálculo de interés** todavía funcionaría. Una operación polimórfica se puede declarar sin una implementación en una clase padre con objeto de que se proporcione una implementación para cada clase descendiente. Tal operación incompleta es abstracta y se representa poniendo su nombre en cursiva.

El otro propósito de la generalización es permitir la descripción incremental de un elemento que comparte las descripciones de sus antecesores. Esto se llama herencia. La herencia es el mecanismo por el cual la descripción de los objetos de una clase se ensambla a partir de los fragmentos de declaración de la clase y de sus antecesores. La herencia permite que las partes compartidas de la descripción sean declaradas una vez y compartidas por muchas clases, en lugar de que se repitan en cada clase que las utiliza. Al compartir se reduce el tamaño del modelo. Más importante aún, reduce el número de los cambios que se deben realizar en una actualización del modelo y reduce la posibilidad de inconsistencia accidental. La herencia trabaja de una manera similar con otras clases de elementos, tales como estados, señales, y casos de uso.

Herencia

Cada clase de elemento generalizable tiene un conjunto de propiedades que se puede heredar. Para cualquier elemento del modelo, éstas incluyen restricciones. Para los clasificadores, también incluyen las propiedades (atributos, operaciones, y recepción de señales) y la participación en asociaciones. Un hijo hereda todas las propiedades heredables de todos sus antecesores. Su conjunto completo de propiedades es el conjunto de propiedades heredadas junto con las propiedades que declara directamente.

En un clasificador, ningún atributo con la misma signature se puede declarar más de una vez (directamente o heredado). De lo contrario, hay un conflicto y el modelo está mal formado. Es decir, un atributo declarado en un antecesor no puede estar redeclarado en un descendiente. Una operación se puede declarar en más de un clasificador, con tal que las especificaciones sean consistentes (los mismos parámetros, restricciones, y significados).

Las declaraciones adicionales son simplemente redundantes. Un método puede ser declarado por múltiples clases en una jerarquía. Un método unido a un descendiente reemplaza y sustituye (elimina) a un método con la misma signature declarado en cualquier antecesor. Sin embargo, si dos o más copias distintas de un método son heredadas por una clase (vía herencia múltiple de diversas clases), entonces están en conflicto y el modelo está mal formado. (Algunos lenguajes de programación permiten elegir uno de los métodos explícitamente. Encontramos más simple y más seguro tener que redefinir el método en la clase hija.) Las restricciones en un elemento son la unión de las restricciones en el propio elemento y todos sus antecesores; si cualesquiera de ellas son contrarias, entonces el modelo está mal formado.

En una clase concreta, cada operación heredada o declarada debe tener un método definido, directamente o por herencia de un antecesor.

Herencia múltiple

Si un clasificador tiene más de un padre, hereda de ambos (Figura 4-8). Sus propiedades (atributos, operaciones, y señales) son la unión de los de sus padres. Aunque la misma clase

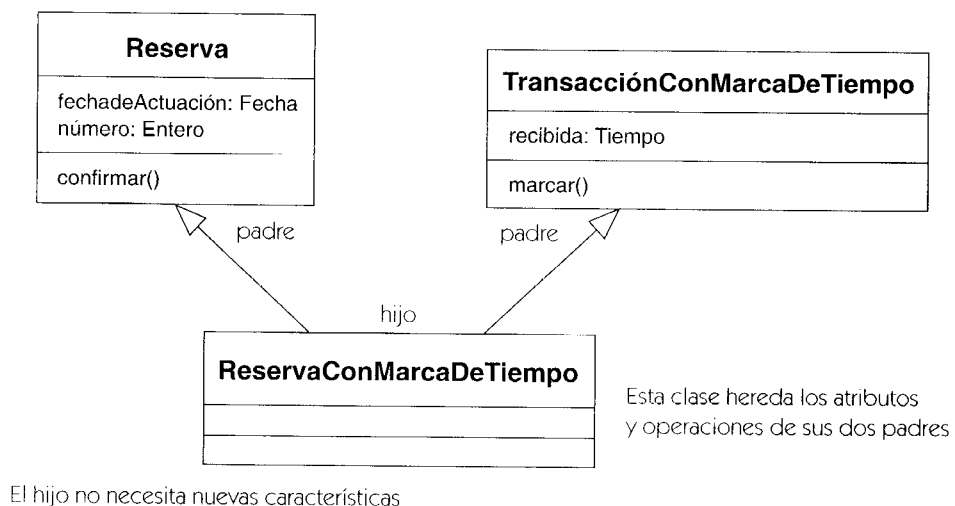


Figura 4.8 Herencia múltiple

aparezca como antecesor por más de una ruta, sólo contribuye con una copia de cada uno de sus miembros. Si una propiedad, con la misma signatura, es declarada por dos clases que no la heredan de un antecesor común (declaraciones independientes), entonces las declaraciones están en conflicto y el modelo está mal formado. UML no proporciona una regla de resolución del conflicto, para esta situación, porque la experiencia ha demostrado que el diseñador debe resolverla explícitamente. Algunos lenguajes, como Eiffel, permiten que los conflictos sean resueltos explícitamente por el programador, que es mucho más seguro que las reglas implícitas de resolución del conflicto, que conducen con frecuencia a sorpresas para el desarrollador.

Clasificación simple y múltiple

En la formulación más simple, un objeto tiene una clase directa. Muchos lenguajes orientados a objetos tienen esa restricción. No hay necesidad lógica de que un objeto tenga una sola clase; nosotros observamos casi siempre los objetos del mundo real desde muchos ángulos simultáneamente. En la formulación más general de UML, un objeto puede tener una o más clases directas. El objeto se comporta como si perteneciera a una clase implícita que fuera hija de cada una las clases directas; en este sentido, con herencia múltiple no hay necesidad realmente de declarar la nueva clase.

Clasificación estática y dinámica

En la formulación más simple, un objeto no puede cambiar su clase después de haber sido creado. Una vez más, no hay necesidad lógica para esta restricción. Tiene como objeto sobre todo, hacer la implementación de lenguajes de programación orientados a objetos más sencilla. En la formulación más general, un objeto puede cambiar su clase directa dinámicamente. Haciendo eso, puede perder o ganar atributos o asociaciones. Si los pierde, la información en ellas se pierde y no se puede recuperar más adelante, incluso si cambia de nuevo a la clase original. Si gana atributos o asociaciones, entonces deben ser inicializados en el momento del cambio, de manera similar a la inicialización de un nuevo objeto.

Cuando la clasificación múltiple se combina con la clasificación dinámica, un objeto puede ganar y perder clases durante toda su vida. Las clases dinámicas a veces se llaman roles o tipos. Un patrón de modelado común es requerir que cada objeto tenga una sola clase inherente estática (una que no pueda cambiar durante toda la vida de objeto) y cero o más clases rol que se puedan agregar o quitar durante el curso de vida del objeto. La clase inherente describe sus propiedades fundamentales, y las clases de rol describen propiedades transitorias. Aunque muchos lenguajes de programación no apoyan la clasificación dinámica múltiple en la jerarquía de declaración de clases es, sin embargo, un concepto de modelado valioso, que se puede representar con asociaciones.

Realización

La relación de realización conecta un elemento del modelo, tal como una clase, con otro elemento, tal como una interfaz, que especifica su comportamiento pero no su estructura o implementación. El cliente debe tener (por herencia o por declaración directa), al menos todas las

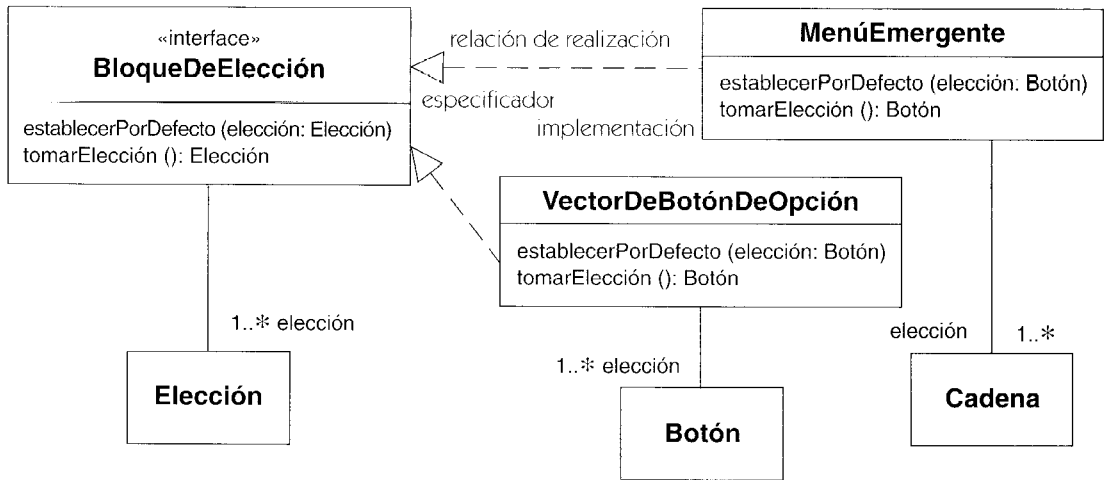


Figura 4.9 Relación de realización

operaciones que tiene el proveedor. Aunque la realización pretende ser utilizada con elementos de especificación, tales como interfaces, también puede ser utilizada con un elemento concreto de la implementación para indicar que su especificación (pero no su implementación) debe estar presente. Por ejemplo, esto se puede utilizar para mostrar la relación de una versión optimizada de una clase a una versión más simple pero ineficaz.

La generalización y la realización relacionan una descripción más general, con versiones más detalladas de la misma. La generalización relaciona dos elementos en el mismo nivel semántico (en el mismo nivel de abstracción, por ejemplo), generalmente dentro del mismo modelo; la realización relaciona dos elementos en diversos niveles semánticos (una clase del análisis y una clase de diseño, por ejemplo, o una interfaz y una clase), a menudo en diversos modelos. Puede haber dos o más jerarquías completas de clases en distintas etapas de desarrollo, cuyos elementos están relacionados por realización. Las dos jerarquías no necesitan tener la misma forma, porque las clases que la realizan pueden tener dependencias de implementación que no sean relevantes en las clases que actúan de especificación.

La realización se indica con una flecha de línea discontinua con una punta de flecha hueca cerrada (Figura 4.9). Es similar al símbolo de la generalización con una línea discontinua, para indicar que es similar a un tipo de herencia.

Hay una notación reducida especial para mostrar interfaces (sin su contenido) y las clases o los componentes que los realizan. Se muestra la interfaz como un círculo pequeño, unido al rectángulo del clasificador por una línea sólida (Figura 4.10).



Figura 4.10 Interfaze iconos de realización

Dependencias

Una dependencia indica una relación semántica entre dos o más elementos del modelo. Relaciona los elementos del modelo entre ellos, y no requiere un conjunto de instancias para su significado. Indica una situación, en la cual un cambio al elemento proveedor puede requerir un cambio o indicar un cambio en el significado del elemento cliente en la dependencia.

Las relaciones de asociación y generalización son dependencias según esta definición, pero tienen semántica específica con consecuencias importantes. Por lo tanto, tienen sus propios nombres y semántica detallada. Utilizamos normalmente la palabra dependencia para el resto de relaciones, que no encajan en categorías más definidas. La Tabla 4.3 enumera las clases de dependencia encontradas en el modelo base de UML.

Una traza es una conexión conceptual entre elementos de diversos modelos, a menudo modelados en diferentes etapas del desarrollo. Carece de semántica detallada. Se utiliza típicamente para seguir la pista de requisitos del sistema, a través de los modelos, y para no perder de vista los cambios realizados en un modelo, que puedan afectar otros modelos.

Un refinamiento es una relación entre dos versiones de un concepto en diversas etapas del desarrollo o en diversos niveles de abstracción. Los dos conceptos no pretenden coexistir en el modelo final detallado. Uno de ellos es generalmente una versión menos acabada del otro. En principio, existe una correspondencia del concepto menos acabado al concepto acabado. Esto no significa que la traducción sea automática. Generalmente, el concepto más detallado contiene las decisiones tomadas por el diseñador, las decisiones del diseño que podrían haberse tomado de muchas maneras. En principio, los cambios a un modelo se podrían validar contra el otro, señalando las desviaciones. En la práctica, las herramientas no pueden hacer todo esto hoy en día, aunque se pueden implementar algunas correspondencias más sencillas. Por lo tanto un refinamiento es sobre todo un recordatorio al modelador de la presencia de múltiples modelos relacionados de una manera previsible.

Una dependencia de derivación indica que un elemento se puede computar a partir de otro elemento (se puede haber incluido explícitamente el elemento derivado en el sistema para evitar una recomputación costosa). La derivación, la realización, el refinamiento, y la traza son dependencias de abstracción; relacionan dos versiones de la misma cosa subyacente.

Una dependencia de uso es una declaración de que el comportamiento o la implementación de un elemento, afectan al comportamiento o a la implementación de otro elemento. Con frecuencia, esto se debe a temas de implementación, tales como requisitos del compilador que necesita la definición de una clase para compilar otra clase. La mayoría de las dependencias de uso se pueden derivar del código y no necesitan ser declaradas explícitamente, a menos que sean parte de un estilo de diseño descendente, que limita la organización del sistema (por ejemplo, usando componentes y bibliotecas predefinidos). El tipo específico de dependencia de uso puede ser especificado, pero esto se omite a menudo porque el propósito de la relación es destacar la dependencia. Los detalles exactos se pueden obtener a menudo del código de implementación. Los estereotipos de uso incluyen la llamada y la instanciación. La dependencia de llamada indica que un método, en una clase, llama a una operación en otra clase; la instanciación indica que un método, en una clase, crea una instancia de otra clase.

Algunas variedades de dependencia de uso conceden permiso para que los elementos tengan acceso a otros elementos. La dependencia de acceso permite que un paquete vea el contenido de otro paquete. La dependencia de importación va más lejos, y agrega los nombres de los contenidos del paquete destino al espacio de nombres del paquete desde el que se importa. La de-

Tabla 4.3 Tipos de Dependencias

<i>Dependencia</i>	<i>Función</i>	<i>Palabra clave</i> ¹
acceso	Permiso para un paquete para acceder a los contenidos de otro paquete	access/accede
ligadura	Asignación de valores a los parámetros de una plantilla, para generar un nuevo elemento de modelo	bind*/ligado
llamada	Establece que un método de una clase llama a una operación de otra	call/llama
derivación	Establece que una instancia puede ser calculada a partir de otra	derive/deriva
amigo	Permiso para un elemento para acceder a otro elemento referente a la visibilidad	friend/amiga
importación	Permiso para un paquete para acceder a los contenidos de otro paquete y añadir alias de sus nombres en el espacio de nombres del importador	import/importa
instanciación	Establece que un método de una clase crea instancias de otra clase	instantiate/usa instancias
parámetro	Relación entre una operación y sus parámetros	parameter/parámetro
realización	Correspondencia entre una especificación y la implementación de la misma	realize/realiza
refinamiento	Establece que existe una correspondencia entre elementos a dos niveles semánticos diferentes	refine*/refina
envío	Relación entre el emisor de una señal y el receptor de la misma	send/envía
traza	Establece que existe alguna conexión entre elementos en diferentes modelos, pero menos preciso que una correspondencia	trace*/traza
uso	Establece que un elemento requiere la presencia de otro elemento, para su correcto funcionamiento (incluye llamada, instanciación, parámetro, envío, pero está abierto a otros tipos)	use*/usa

pendencia de amistad es una dependencia de acceso, que permite que el cliente vea incluso el contenido privado del proveedor.

Una ligadura es la asignación de valores a los parámetros de una plantilla. Es una relación altamente estructurada, con la semántica exacta obtenida sustituyendo los argumentos por los parámetros, en una copia de la plantilla.

¹ Estas palabras clave son estereotipos (véase capítulo 11) y por tanto pueden estar en cualquier idioma. La forma inglesa puede ser la empleada por algunas herramientas (las marcadas con *) y la que aparecerá en modelos UML en lengua inglesa. Pero las formas traducidas pueden utilizarse para que un modelo no tenga que hacer uso de términos ingleses, que podrían oscurecer la comprensión (*N. del Revisor*.)

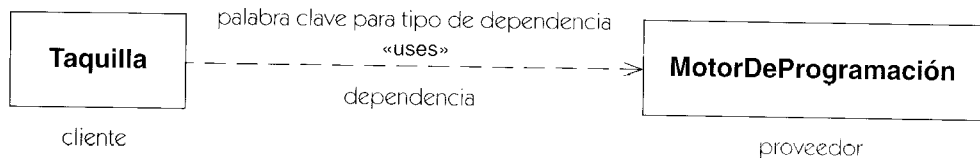


Figura 4.11 Dependencias

Las dependencias de uso y de ligadura implican una semántica fuerte entre elementos al mismo nivel semántico. Deben conectar elementos al mismo nivel del modelo (ambos en análisis o en diseño, y al mismo nivel de abstracción). Las dependencias de traza y de refinamiento son más vagas, y pueden conectar elementos de distintos modelos o niveles de abstracción.

La instancia de la relación (una metarrelación, no estrictamente una dependencia) indica que un elemento (tal como un objeto) es una instancia de otro elemento (tal como una clase).

Una dependencia se dibuja como una flecha de línea discontinua, desde el cliente al proveedor, con una palabra clave de estereotipo para distinguirla, según lo mostrado en la Figura 4.11.

Restricción

UML provee un sistema de conceptos y de relaciones para modelar sistemas como grafos de elementos de modelado. Sin embargo, algunas cosas se expresan mejor lingüísticamente, es decir, usando la potencia de un lenguaje textual. Una restricción es una expresión booleana representada como una cadena interpretable en un determinado lenguaje. Para expresar restricciones se puede utilizar el lenguaje natural, notación de teoría de conjuntos, lenguajes de restricciones o varios lenguajes de programación.

UML incluye la definición de un lenguaje de restricción, llamado OCL, adecuado para expresar restricciones de UML. Esperamos que reciba un amplio apoyo y soporte. Véase la referencia de OCL y el libro [Warmer-99] para más información sobre el mismo.

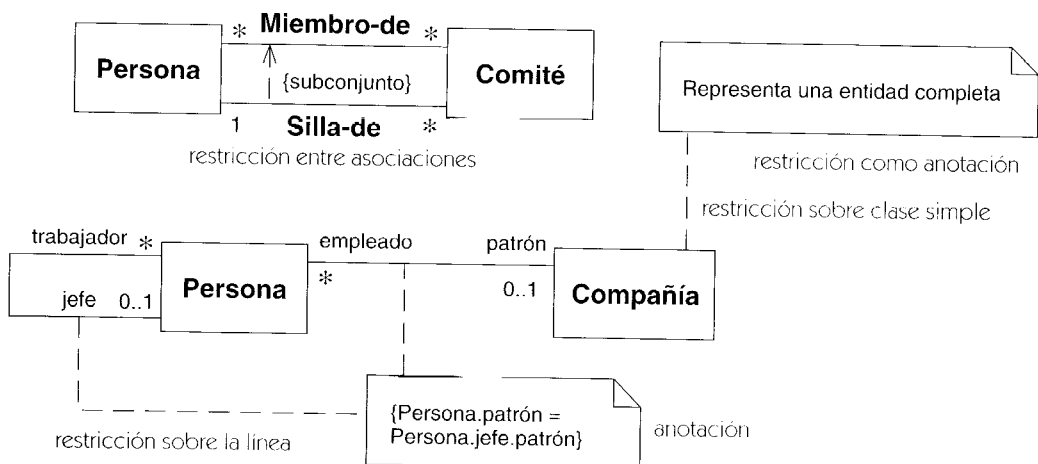


Figura 4.12 Restricciones

Se pueden utilizar las restricciones para indicar varias relaciones no locales, tales como restricciones en las asociaciones. En detalle, las restricciones se pueden utilizar para indicar propiedades de existencia (*existe un X tal que la condición C es verdadera*) y propiedades universales (*para toda y en Y, la condición D debe ser verdadera*).

Algunas restricciones estándar han sido predefinidas como elementos estándar de UML, por ejemplo las asociaciones en una relación de “o exclusivo” y varias restricciones en las relaciones entre subclases en la generalización.

Véase el capítulo 14, elementos estándar, para más información.

Una restricción se muestra como expresión del texto entre llaves. Puede ser escrito en un lenguaje formal o natural. La cadena de texto se puede colocar en una nota o unir a una flecha de dependencia. La Figura 4.12 muestra algunas restricciones.

Instancias

Una instancia es una entidad de ejecución con identidad, es decir, algo que puede ser distinguido de otras entidades de ejecución. Tiene un valor en todo momento. A lo largo del tiempo, el valor puede cambiar en respuesta a operaciones sobre él.

Un propósito de un modelo es describir los posibles estados de un sistema y de su comportamiento. Un modelo es una declaración de potencial, de las posibles colecciones de objetos que pueden existir, y de la posible historia de comportamiento que los objetos pueden experimentar. La vista estática define y restringe las posibles configuraciones de los valores que un sistema en ejecución puede asumir.

La vista dinámica define las maneras por las cuales un sistema en ejecución puede pasar de una configuración a otra. La vista estática, junto a las distintas vistas dinámicas basadas en ella, definen la estructura y el comportamiento de un sistema.

Una configuración estática particular de un sistema en un instante se llama una instantánea. Una instantánea abarca objetos y otras instancias, valores, y enlaces. Un objeto es una instancia de una clase. Cada objeto es una instancia directa de la clase que lo describe totalmente y una instancia indirecta de los antecesores de esa clase. (Si se permite la clasificación múltiple, entonces un objeto puede ser la instancia directa de más de una clase.) Del mismo modo, cada enlace es una instancia de una asociación, y cada valor es una instancia de un tipo de datos.

Un objeto tiene un valor para cada atributo de su clase. El valor de cada atributo debe ser consistente con el tipo de dato del atributo. Si el atributo tiene multiplicidad opcional o múltiple, entonces el atributo puede contener cero o múltiples valores. Un enlace abarca una tupla de valores, cada uno de los cuales es una referencia a un objeto de una clase dada (o uno de sus descendientes). Los objetos y los enlaces deben obedecer cualquier restricción en las clases o las asociaciones de las cuales son instancias (incluyendo tanto restricciones explícitas como restricciones incorporadas, tales como la multiplicidad).

El estado de un sistema es una *instancia válida del sistema* si cada instancia en él es instancia de algún elemento de un mismo modelo de sistema, este modelo está bien formado, y todas las restricciones impuestas por el modelo son satisfechas por los casos.

La vista estática define el conjunto de objetos, valores, y enlaces que pueden existir en una sola instantánea. En principio, cualquier combinación de objetos y de enlaces que sea consistente

con una vista estática es una configuración posible del modelo. Esto no significa que toda instantánea posible pueda ocurrir u ocurra. Algunas instantáneas pueden ser legales estáticamente, pero pueden no ser dinámicamente accesibles bajo las vistas dinámicas del sistema.

Los elementos y diagramas para comportamiento de UML describen las secuencias válidas de las instantáneas que pueden ocurrir como resultado de efectos del comportamiento externo e interno. Las vistas dinámicas definen cómo se mueve el sistema de una instantánea a otra.

Diagrama de objetos

Un diagrama de una instantánea es una imagen de un sistema, en un instante en el tiempo. Debido a que contiene imágenes de objetos, se llama diagrama de objetos. Puede ser útil como ejemplo del sistema, por ejemplo, ilustrar las estructuras de datos complicadas o mostrar el comportamiento con una secuencia de instantáneas en un cierto plazo (Figura 4.13). Recuerde que todas las instantáneas son ejemplos de sistemas, no definiciones de sistemas. La definición de la estructura y del comportamiento del sistema se encuentra en las vistas de definición, y construir las vistas de definición es el objetivo del modelado y el diseño.

La vista estática describe los casos posibles que pueden ocurrir. Los casos reales generalmente no aparecen directamente en modelos, excepto como ejemplos.

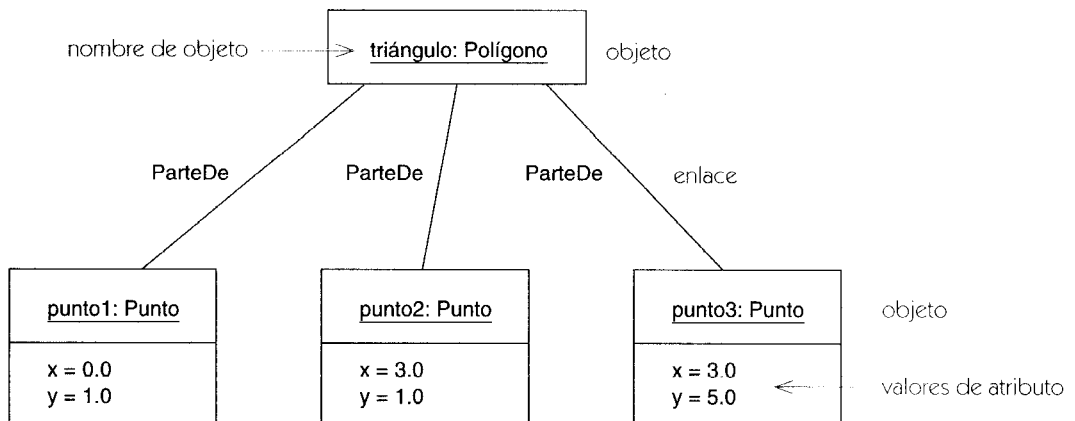


Figura 4.13 Diagrama de objetos

Descripción

La vista de casos de uso captura el comportamiento de un sistema, de un subsistema, o de una clase, tal como se muestra a un usuario exterior. Reparte la funcionalidad del sistema en transacciones significativas para los actores-usuarios ideales de un sistema. Las piezas de funcionalidad interactiva se llaman casos de uso. Un caso de uso describe una interacción con los actores como secuencia de mensajes entre el sistema y uno o más actores. El término actor incluye a los seres humanos, así como a otros sistemas informáticos y procesos. La Figura 5.1 muestra un diagrama de casos de uso, para una aplicación telefónica de venta por catálogo. El modelo se ha simplificado como ejemplo.

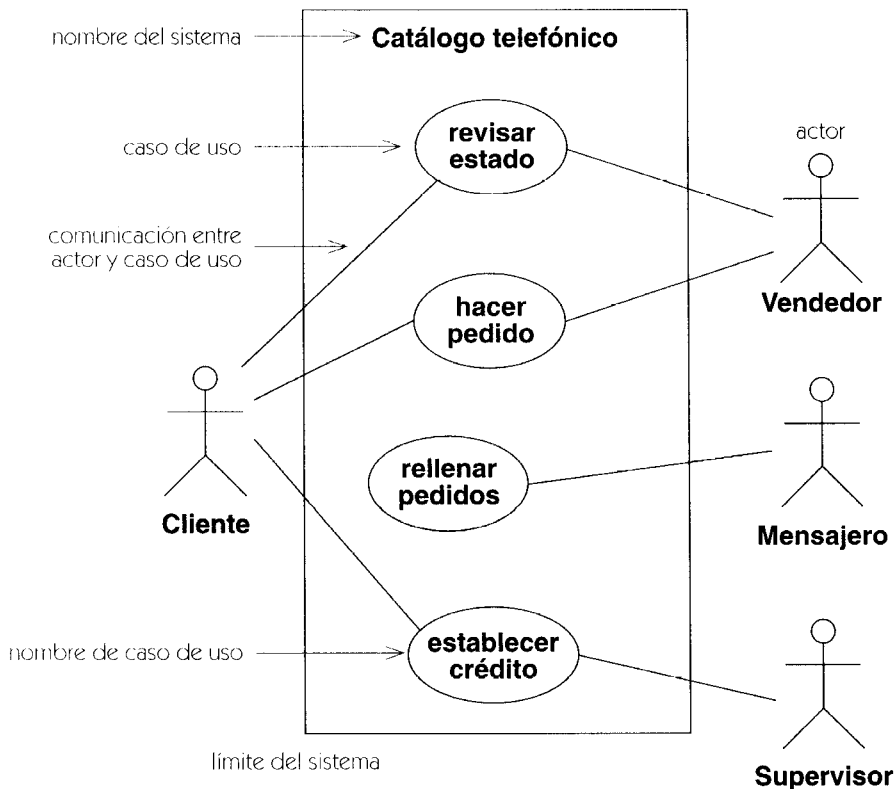


Figura 5.1 Diagrama de casos de uso

Actor

Un actor es una idealización de una persona externa, de un proceso, o de una cosa que interactúa con un sistema, un subsistema, o una clase. Un actor caracteriza las interacciones que los usuarios exteriores pueden tener con el sistema. En tiempo de ejecución, un usuario físico puede estar limitado a los actores múltiples dentro del sistema. Diferentes usuarios pueden estar ligados al mismo actor y por lo tanto pueden representar casos múltiples de la misma definición de actor.

Cada actor participa en uno o más casos de uso. Interactúa con el caso de uso (y por lo tanto con el sistema o la clase que posee el caso de uso), intercambiando mensajes. La implementación interna de un actor no es relevante en el caso de uso; un actor puede ser caracterizado suficientemente por un conjunto de atributos que definen su estado.

Los actores pueden ser definidos en jerarquías de generalización, en las cuales una descripción abstracta del actor es compartida y aumentada por una o más descripciones específicas del actor.

Un actor puede ser un ser humano, otro sistema informático, o un cierto proceso ejecutable.

Se dibuja a un actor como una persona pequeña con trazos lineales y el nombre debajo de él.

Caso de uso

Un caso de uso es una unidad coherente de funcionalidad, externamente visible, proporcionada por una unidad del sistema y expresada por secuencias de mensajes intercambiados por la unidad del sistema y uno o más actores. El propósito de un caso de uso es definir una pieza de comportamiento coherente, sin revelar la estructura interna del sistema. La definición de un caso de uso incluye todo el comportamiento que implica: las líneas principales, las diferentes variaciones sobre el comportamiento normal, y todas las condiciones excepcionales, que pueden ocurrir con tal comportamiento, junto con la respuesta deseada. Desde el punto de vista de los usuarios, éstas pueden ser situaciones anormales. Desde el punto de vista de los sistemas, son las variaciones adicionales que deben ser descritas y manejadas.

En el modelo, la ejecución de cada caso de uso es independiente de las demás, aunque una implementación de casos de uso puede crear dependencias implícitas entre ellas, debido a los objetos compartidos. Cada caso de uso representa una pieza ortogonal de la funcionalidad, cuya ejecución se puede mezclar con la ejecución de otros casos de uso.

La dinámica de un caso de uso se puede especificar por las interacciones de UML, mostradas como diagramas de estado, diagramas de secuencia, diagramas de colaboración, o descripciones informales de texto. Cuando se implementan, los casos de uso son realizados mediante colaboraciones entre clases del sistema. Una clase puede participar en múltiples colaboraciones y, por lo tanto, en múltiples casos de uso.

En el nivel de sistema, los casos de uso representan el comportamiento externo de todo sistema tal y como se muestra a los usuarios exteriores. Un caso de uso es como una operación de sistema, una operación invocable por un usuario exterior. Sin embargo, a diferencia de una operación, un caso de uso puede continuar recibiendo la entrada de sus actores durante su ejecución. Los casos de uso también se pueden aplicar internamente a unidades más pequeñas de un sistema, tales como subsistemas y clases individuales. Un caso interno de uso representa el comportamiento que una parte del sistema presenta al resto del sistema. Por ejemplo, un caso

de uso para una clase representa una pieza coherente de la funcionalidad que una clase proporciona a otras clases que desempeñen ciertos roles dentro del sistema. Una clase puede tener más de un caso de uso.

Un caso de uso es una descripción lógica de una parte de funcionalidad del sistema. No es una construcción manifiesta en la implementación de un sistema. En su lugar, cada caso de uso se debe corresponder con las clases que implementan un sistema. El comportamiento del caso de uso se corresponde con las transiciones y operaciones de las clases. Ya que una clase puede desempeñar roles múltiples en la implementación de un sistema, puede por lo tanto realizar porciones de múltiples casos de uso. Una parte de la tarea del diseño es encontrar las clases de implementación que combinen claramente los roles apropiados para implementar todos los casos de uso, sin introducir complicaciones innecesarias. La implementación de un caso de uso se puede modelar como un conjunto de una o más colaboraciones. Una colaboración es una realización de un caso de uso.

Un caso de uso puede participar en varias relaciones, además de poderse asociar con actores (Tabla 5.1).

Un caso de uso se dibuja como una elipse con su nombre dentro o debajo de ella. Se conecta por líneas con trazo continuo con los actores que se comunican con ella.

Aunque cada instancia de un caso de uso es independiente, la descripción de un caso de uso se puede descomponer en factores de otros casos de uso más simples. Esto es similar a la manera en que la descripción de una clase se puede definir incrementalmente a partir de la descripción de las superclases. Un caso de uso puede incorporar simplemente el comportamiento de otros casos de uso como fragmentos de su propio comportamiento. Esto se llama relación de inclusión. En este caso, el nuevo caso de uso no es un caso especial del caso de uso original, y no se puede sustituir por él.

Un caso de uso se puede también definir como una extensión incremental de un caso de uso base. Esto se llama relación de extensión. Puede haber varias extensiones del mismo caso de uso base, que pueden ser aplicadas conjuntamente. Las extensiones a un caso de uso base se agregan a su semántica; que es el caso de uso base instanciado, no los casos de uso de la extensión.

Las relaciones de inclusión y extensión se dibujan como flechas de líneas discontinuas con la palabra clave «**include**» o «**extend**». La relación de inclusión apunta al caso de uso a ser incluido; la relación de extensión señala el caso de uso que se extenderá.

Tabla 5.1 Tipos de Relaciones de Casos de Uso

<i>Relación</i>	<i>Función</i>	<i>Notación</i>
asociación	La línea de comunicación entre un actor y un caso de uso en el que participa	_____
extensión	La inserción de comportamiento adicional en un caso de uso base que no tiene conocimiento sobre él	« extend » - - - ➤
generalización de casos de uso	Una relación entre un caso de uso general y un caso de uso más específico, que hereda y añade propiedades a aquél	—————▶
inclusión	Inserción de comportamiento adicional en un caso de uso base, que describe explícitamente la inserción	« include » - - - ➤

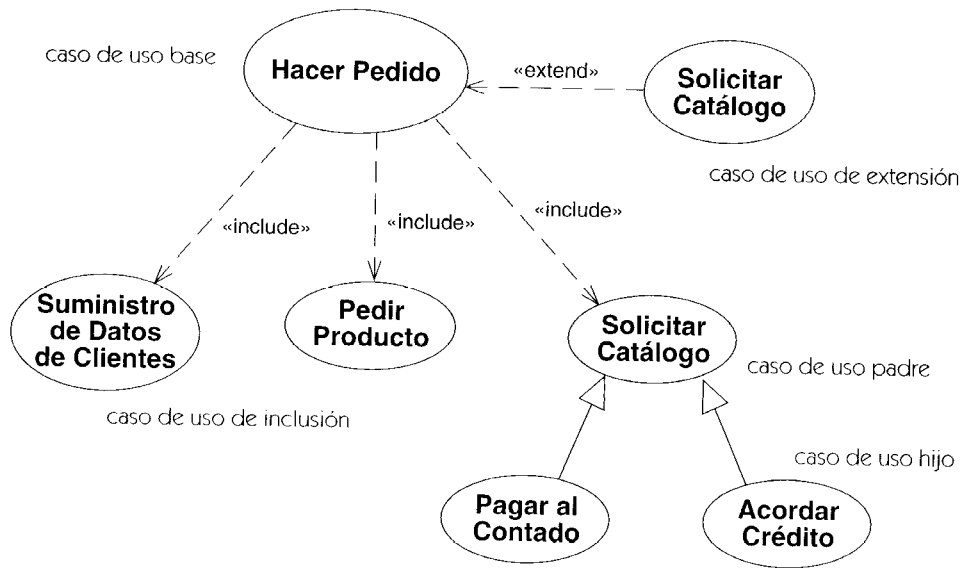


Figura 5.2 Relaciones de casos de uso

Un caso de uso también se puede especializar en uno o más casos de uso hijos. Ésta es la generalización de casos de uso. Cualquier caso de uso hijo se puede utilizar en una situación en la cual se espera el caso de uso padre.

La generalización de casos de uso se dibuja igual que cualquier generalización, utilizando una línea desde el caso de uso hijo al caso de uso padre con una punta de flecha triangular grande en el extremo del padre. La Figura 5.2 muestra las relaciones de casos de uso en la aplicación de venta por catálogo.

La vista de la máquina de estados

Descripción

La vista de la máquina de estados describe el comportamiento dinámico de los objetos, en un cierto plazo, modelando los ciclos de vida de los objetos de cada clase. Cada objeto se trata como una entidad aislada que se comunica con el resto del mundo detectando eventos y respondiendo a ellos. Los eventos representan las clases de cambios que un objeto puede detectar: la recepción de llamadas o señales explícitas desde un objeto a otro, un cambio en ciertos valores, o el paso del tiempo. Cualquier cosa que pueda afectar a un objeto se puede caracterizar como evento. Los sucesos del mundo real se modelan como señales del mundo exterior al sistema.

Un estado es un conjunto de los valores de un objeto para una clase dada, que tienen la misma respuesta cualitativa a los eventos que ocurren. Es decir, todos los objetos con el mismo estado reaccionan de la misma manera general a un evento, así que todos los objetos en un estado dado ejecutan la misma acción cuando reciben el mismo evento. Sin embargo, los objetos en diferentes estados, pueden reaccionar de diferente forma al mismo evento, realizando distintas acciones. Por ejemplo, una máquina de cajero automático reacciona de una forma al botón de cancelar cuando está procesando una transacción y de otra manera cuando está inactiva.

Las máquinas de estados describen el comportamiento de las clases, pero también describen el comportamiento dinámico de los casos de uso, de las colaboraciones, y de los métodos. Para uno de estos objetos, un estado representa un paso en su ejecución. Cuando hablamos de las máquinas de estado como descripción lo hacemos, la mayoría de las veces, en términos de clases y objetos, pero pueden ser aplicadas a otros elementos de manera directa.

Máquina de estados

Una máquina de estados es un gráfico de estados y de transiciones. Una máquina de estados se une a una clase y describe, generalmente, la respuesta de una instancia de la clase, a los eventos que recibe. Las máquinas de estados también se pueden unir a las operaciones, a los casos de uso, y a las colaboraciones para describir su ejecución.

Una máquina de estados es un modelo de todas las posibles historias de vida de un objeto de una clase. El objeto se examina aisladamente. Cualquier influencia externa del resto del mundo se resume como evento. Cuando el objeto detecta un evento, responde de una manera que depende de su estado actual. La respuesta puede incluir la ejecución de una acción y un cambio a un nuevo estado. Las máquinas de estados se pueden estructurar para heredar transiciones, y pueden modelar la concurrencia.

Una máquina de estados es una vista localizada de un objeto, una vista que lo separe del resto del mundo y examine su comportamiento aislado. Es una vista reduccionista de un sistema. Es una buena manera de especificar un comportamiento exacto, pero no es, a menudo, una buena manera de entender el funcionamiento total de un sistema. Para una idea más amplia de los efectos del comportamiento de un sistema, son más útiles las vistas de interacción. Sin embargo, las máquinas de estados son útiles para entender los mecanismos de control, tales como interfaces de usuario o controladores de dispositivo.

Evento

Un evento es una ocurrencia significativa que tiene una localización en tiempo y espacio. Ocurre en un punto en el tiempo y no tiene duración. Modela algo como un evento si su ocurrencia tiene consecuencias. Cuando utilizamos la palabra evento por sí mismo, queremos decir generalmente que es un descriptor, es decir una descripción de todas las ocurrencias individuales del evento, que tienen la misma forma general, del mismo modo que la palabra clase expresa todos los objetos individuales, que tienen la misma estructura. Una ocurrencia específica de un evento se llama instancia del evento. Los eventos pueden tener parámetros que caractericen cada instancia individual del evento, de la misma forma que las clases tienen atributos que caracterizan cada objeto. Como con las clases, las señales se pueden ordenar en jerarquías de generalización, para compartir la estructura común. Los eventos se pueden dividir en varios tipos explícitos e implícitos: eventos de señal, eventos de llamada, eventos de cambio y eventos de tiempo. La Tabla 6.1 es una lista de los tipos de eventos y sus descripciones.

Tabla 6.1 Tipos de Eventos

<i>Tipo de evento</i>	<i>Descripción</i>	<i>Sintaxis</i>
evento de llamada	Recepción de una petición explícita síncrona entre objetos que esperan por una respuesta	op (a:T)
evento de cambio	Un cambio en el valor de una expresión Booleana	cuando (expresión)
evento de señal	Recepción de una comunicación asíncrona explícita y con nombre entre objetos	nombreS (a:T)
evento de tiempo	La conclusión de un tiempo absoluto o el transcurso de una cantidad relativa de tiempo	tras (tiempo)

Evento de la señal. Una señal es una entidad con nombre, que se piensa explícitamente como vehículo de comunicación entre dos objetos; la recepción de una señal es un evento para el objeto receptor. El objeto que envía, crea explícitamente e inicializa una instancia de la señal y la envía a uno o a un conjunto de objetos explícitos. Las señales incorporan la comunicación unidireccional asíncrona, el tipo más importante. El remitente no espera a que el receptor se ocupe de la señal, sino que continúa con su propio trabajo independientemente. Para modelar la comunicación bidireccional se pueden utilizar varias señales, por lo menos una en cada dirección. El remitente y el receptor pueden ser el mismo objeto.

Las señales se pueden declarar en diagramas de clase como clasificadores, usando la palabra clave «**signal/señal**»¹; los parámetros de la señal se declaran como atributos. Como clasificadores, las señales pueden tener relaciones de generalización. Las señales pueden ser hijas de

¹ Véase nota al pie pág. 51.

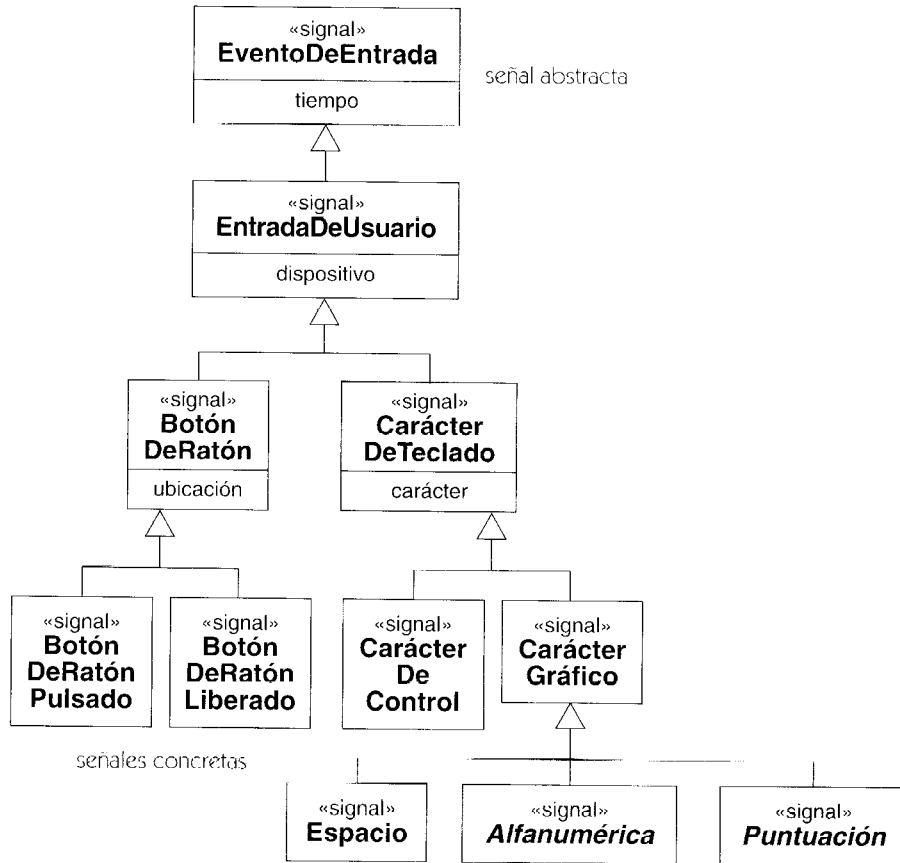


Figura 6.1 Jerarquía de señales

otras señales; heredando los parámetros de sus padres, y accionando las transiciones que dependen de la señal del padre (Figura 6.1).

Evento de llamada. Un evento de llamada es la recepción de una llamada por un objeto que elige poner una operación en ejecución, como transición de la máquina de estados en vez de como procedimiento fijo. Para el llamador, una llamada ordinaria (implementada por un método) es indistinguible de un evento de llamada. El receptor elige si una operación será implementada como un método o como disparador de un evento de llamada en una máquina de estados. Los parámetros de la operación son los parámetros del evento. Una vez que el objeto receptor procesa el evento de llamada tomando una transición accionada por el evento, o no pudiendo tomar ninguna transición, el control retorna al objeto llamador.

Sin embargo, a diferencia de una llamada ordinaria, el receptor de un evento de llamada puede continuar su propia ejecución en paralelo al llamador.

Evento de cambio. Un evento de cambio es la satisfacción de una expresión booleana que dependa de ciertos valores de un atributo. Esto es una manera declarativa de esperar hasta que una condición esté satisfecha, pero debe ser utilizada con cuidado, porque representa un cómputo continuo y potencialmente no local (acción en una distancia, porque el valor o los valores comprobados pueden ser distantes). Esto es bueno y malo. Es bueno porque centra el modelo en

la verdadera dependencia —un efecto que ocurre cuando se satisface una condición dada— y no en la mecánica de comprobación de condición. Es malo porque oscurece la relación causa-efecto entre la acción que cambia un valor subyacente y el efecto eventual. El coste de comprobar un evento de cambio es potencialmente grande, porque en principio es continuo. Sin embargo, hay en la práctica maneras de evitar el cómputo innecesario. Los eventos de cambio deben ser utilizados solamente cuando sea artificial una forma más explícita de comunicación.

Observe la diferencia entre una condición de guarda y un evento de cambio. Una condición de guarda se evalúa una vez cuando ocurre el evento disparador en la transición y el receptor maneja el evento. Si es falsa, la transición no se activa y la condición no se evalúa de nuevo. Un evento de cambio se evalúa continuamente hasta que llega a ser verdad, en cuyo caso se dispara la transición.

Evento de tiempo. Los eventos de tiempo representan el paso del tiempo. Un evento de tiempo se puede especificar de modo absoluto (hora) o de modo relativo (el tiempo transcurrió desde un evento dado). En un modelo de alto nivel, los eventos de tiempo se pueden pensar como eventos del universo; en un modelo de implementación, son causados por las señales de un cierto objeto específico, el sistema operativo o un objeto en la aplicación.

Estado

Un estado describe un período de tiempo durante la vida de un objeto de una clase. Puede ser caracterizado de tres maneras complementarias: como un conjunto de valores de objeto cualitativamente similares en cierta forma; como período de tiempo, durante el cual un objeto espera que ocurra algún evento o eventos; o como período de tiempo, durante el cual un objeto realiza una cierta actividad. Un estado puede tener un nombre, aunque a menudo es anónimo y viene descrito simplemente por sus acciones.

En una máquina de estados, un conjunto de estados está conectado mediante transiciones. Aunque las transiciones conectan dos estados (o más, si hay una división o unión de control), las transiciones son procesadas por el estado del que salen. Cuando un objeto está en un estado, es sensible a los eventos correspondientes a las transiciones que salen del estado.



Confirmar Crédito

Figura 6.2 Estado

Un estado se muestra como rectángulo con las esquinas redondeadas (Figura 6.2).

Transición

Una transición que deja un estado define la respuesta de un objeto en ese estado a la ocurrencia de un evento. En general, una transición tiene un evento que la activa (evento disparador), una condición de guarda, una acción, y un estado destino. La Tabla 6.2 muestra los tipos de las transiciones y de las acciones implícitas invocadas por transiciones.

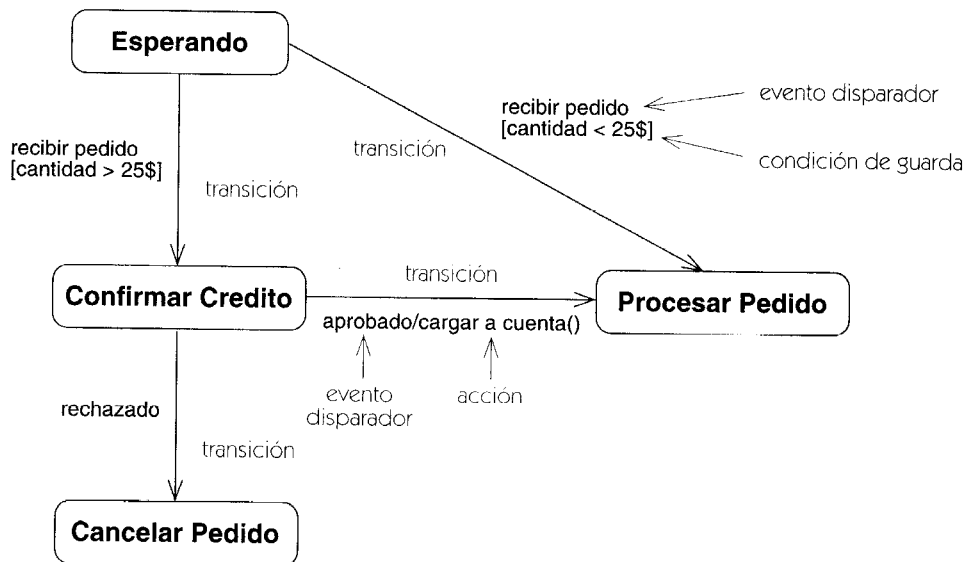
Transición externa. Una transición externa es una transición que cambia el estado activo. Éste es el tipo más común de transición. Se dibuja como una flecha desde el estado origen al estado destino, con otras propiedades mostradas como una cadena de texto asociada a la flecha (Figura 6.3).

Tabla 6.2 Tipos de transiciones y acciones implícitas

Tipo de transición	Descripción	Sintaxis
acción de entrada	Una acción que se ejecuta cuando se entra en el estado	entry/ acción o entrada/ acción
acción de salida	Una acción que se ejecuta cuando se sale del estado	exit/ acción o Salida/ acción
transición externa	Una respuesta a un evento que causa un cambio de estado o una transición a sí mismo, junto con la acción especificada. También puede provocar la ejecución de la acción de entrada o salida en el caso de que haya estados en los que haya acciones asociadas a la entrada o a la salida	$e(a:T)[exp]/$ acción
transición interna	Una respuesta a un evento que causa la ejecución de una acción pero no causa cambio de estado o ejecución de acciones de salida o entrada	$e(a:T)[exp]/$ acción

Evento disparador. El disparador es un evento cuya ocurrencia permite la transición. El evento puede tener parámetros, que estarán disponibles para una acción en la transición. Si una señal tiene descendientes, cualquier descendiente de la señal permite la transición. Por ejemplo, si una transición tiene **BotónDeRatón**, como disparador, (véase la Figura 6.1), entonces **BotónDeRatónPulsado** accionará la transición.

Un evento no es algo continuo; ocurre en un punto en el tiempo. Cuando un objeto recibe un evento, guarda el evento si no es capaz de manejarlo. Un objeto maneja un evento a la vez. Debe activarse una transición cuando el objeto maneja el evento; el evento no es “recordado” hasta más adelante (excepto en el caso especial de los eventos diferidos, que se guardan hasta que accionan una transición o hasta que el objeto está en un estado donde no se difieren). Si ocurren


Figura 6.3 Transiciones externas

dos eventos simultáneamente, se manejan a la vez. Un evento que no acciona ninguna transición es simplemente ignorado y se pierde. Esto no es un error. Es mucho más fácil no hacer caso de los eventos no deseados que intentar especificarlos todos.

Condición de guarda. Una transición puede tener una condición de guarda, que es una expresión booleana. Puede referirse a los atributos del objeto que posee la máquina de estados, así como a los parámetros del evento del disparador. Se evalúa la condición de guarda cuando ocurre un evento disparador. Si la expresión se evalúa como verdadera, entonces se dispara la transición, es decir, ocurren sus efectos. Si la expresión se evalúa como falsa, entonces la transición no se dispara.

La condición de guarda se evalúa solamente una vez: cuando ocurre el evento disparador. Si la condición es falsa y llega a ser más adelante verdad, es demasiado tarde para disparar la transición.

El mismo evento puede ser un disparador para más de una transición que parte de un estado simple. Cada transición con el mismo evento debe tener una condición de guarda diferente. Si ocurre el evento, una transición accionada por el evento puede accionarse si su condición es verdadera. A menudo, el conjunto de condiciones de guarda cubre todas las posibilidades, de tal forma que está garantizado que la ocurrencia de un evento dispara una cierta transición. Si no se cubren todas las posibilidades y no se permite ninguna transición, entonces un evento simplemente se ignora. Una transición únicamente puede dispararse (dentro de un hilo de control) en respuesta a una ocurrencia de un evento. Si un evento permite más de una transición, solamente se dispara una de ellas. Una transición en un estado anidado tiene preferencia sobre una transición de uno de los estados que la incluyen. Si dos transiciones que están en conflicto se activan al mismo tiempo, alguna de ellas se dispara, pero queda indeterminada cuál. La opción puede ser aleatoria o puede depender de los detalles de la implementación, pero el modelador no debe contar con un resultado predecible.

Transición de finalización. Una transición que carece de evento disparador explícito es accionada por la terminación de la actividad del estado del que sale (esto es una transición de terminación). Una transición de terminación puede tener una condición de guarda, que se evalúa en el momento en que termina la actividad en ese estado (y no después).

Acción. Cuando se dispara una transición, su acción (si la hay), es ejecutada. Una acción es un cómputo atómico y normalmente breve, a menudo una declaración de asignación o un cómputo aritmético simple. Otras acciones incluyen enviar una señal a otro objeto, llamar a una operación, fijar los valores de retorno, crear o destruir un objeto, y las acciones indefinidas de control especificadas en un lenguaje externo. Una acción puede también ser una secuencia de acciones, es decir, una lista de acciones más simples. Una acción o una secuencia de acciones no se puede terminar o ser afectada por acciones simultáneas. Conceptualmente, su duración es insignificante comparada con la sincronización exterior del evento; por lo tanto, un segundo evento no puede ocurrir durante su ejecución. En la práctica, sin embargo, las acciones toman un cierto tiempo, y los eventos entrantes se deben poner en una cola.

El sistema total puede realizar múltiples acciones simultáneamente. Cuando decimos acciones atómicas, no implica que todo el sistema sea atómico. El sistema puede procesar interrupciones hardware y compartir el tiempo entre varias acciones. Una acción es atómica dentro de su propio hilo de control. Una vez que ha comenzado, debe terminar y no debe interactuar con otras acciones simultáneamente activas. Pero las acciones no se deben utilizar como mecanismos de transición. Su duración debe ser breve comparada con el tiempo de reacción necesario para los eventos externos. De lo contrario, el sistema puede no responder de manera oportuna.

Tabla 6.3 Tipos de acciones

<i>Tipo de acción</i>	<i>Descripción</i>	<i>Sintaxis²</i>
asignación	Asigna el valor de una variable	destino := expresión
creación	Crea un nuevo objeto	new NombreClase (arg,arg)
destrucción	Destruye un objeto	objeto. destroy ()
enviar	Crea una instancia de una señal y la envía a un objeto destino o conjunto de objetos	Nombres(arg,arg)
llamada	Llama una operación en un objeto destino y espera la terminación de la ejecución de la operación. Puede devolver un valor	nombrcop (arg,arg)
no interpretada	Acción específica del lenguaje, tal como un bucle o condición	[específico del lenguaje]
retorno	Especifica los valores a devolver al llamador	return valor
terminación	Autodestrucción del objeto	terminate

Una acción puede utilizar los parámetros del evento disparador y los atributos del objeto poseedor como parte de su expresión.

La Tabla 6.3 enumera las clases de acciones y sus descripciones.

Cambio de estado. Cuando se completa la ejecución de una acción, el estado destino de la transición pasa a ser el estado activo. Esto puede activar acciones de salida y acciones de entrada.

Estados anidados. Los estados se pueden anidar dentro de otros estados compuestos (véase el punto siguiente). Una transición que deja el estado más externo es aplicable a todos los estados anidados dentro de él. La transición es elegible para ser disparada siempre que cualquier estado anidado esté activo. Si se dispara, el estado destino de la transición pasa a ser el estado activo. Los estados compuestos son útiles para expresar condiciones de excepción y de error, porque las transiciones en ellos se aplican a todos los estados anidados sin necesidad de que cada estado anidado maneje la excepción explícitamente.

Acciones de entrada y salida. Una transición a través de uno o más niveles de anidamiento puede salir y entrar en estados. Un estado puede tener acciones que se realicen siempre que se entre o se salga del estado. Entrando al estado destino, se ejecuta una acción de entrada unida al estado. Si la transición sale del estado original, entonces su acción de salida se ejecuta antes de la acción, de la transición y de la acción de entrada en el nuevo estado.

Las acciones de entrada se utilizan, a menudo, para realizar la configuración necesaria dentro de un estado. Debido a que una acción de entrada no puede ser evadida, cualquier acción que ocurra dentro del estado, puede asumir que la configuración ha ocurrido, sin importar cómo se entre en el estado.

De manera similar, una acción de salida es una acción que ocurre siempre que se salga del estado, una oportunidad de realizar una limpieza. Es particularmente útil cuando hay transiciones

² Salvo si las herramientas usadas necesitan el uso de los términos ingleses, pueden utilizarse los términos traducidos: nuevo, destruir, devolver, terminar. (N. de los R.T.)

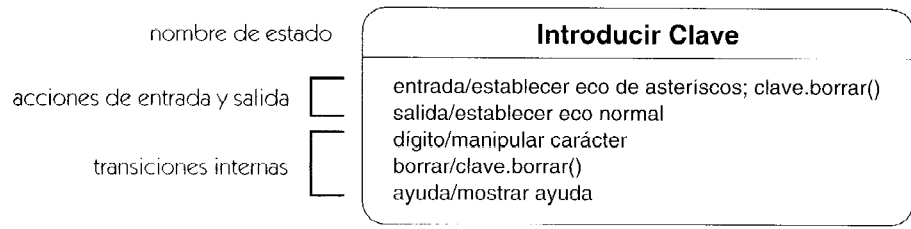


Figura 6.4 Transiciones internas y acciones de entrada y salida

de alto nivel que representan las condiciones de error que abortan estados anidados. La acción de salida puede limpiar tales casos de modo que el estado del objeto siga siendo consistente. Las acciones de entrada y de salida se podrían, en principio, dar como transiciones entrantes y salientes, pero el declararlas como acciones especiales del estado, permite que el estado sea definido independientemente de sus transiciones y por lo tanto esté encapsulado.

Transición interna. Una transición interna tiene un estado origen pero ningún estado destino. Las reglas de disparo para una transición interna son iguales que para una transición que cambie el estado. Una transición interna no tiene ningún estado destino, así que el estado activo no cambia como resultado de su activación. Si una transición interna tiene una acción, se ejecuta, pero no ocurre ningún cambio de estado, y por lo tanto no se ejecuta ninguna de las acciones de salida o de entrada. Las transiciones internas son útiles para modelar las acciones de interrupción que no cambian el estado (como recuento de ocurrencias de un evento o desplegar una pantalla de ayuda).

Las acciones de entrada y de salida utilizan la misma notación que las transiciones internas, pero usan las palabras reservadas **entry** y **exit** (**entrada** y **salida**) en lugar del nombre del evento disparado, aunque estas acciones son activadas por las transiciones externas que entran o salen del estado.

Una transición a sí mismo invoca acciones de salida y de entrada en su estado (conceptualmente, sale y entonces entra al estado de nuevo); por lo tanto, no es equivalente a una transición interna. La Figura 6.4 muestra acciones de entrada y salida así como transiciones internas.

Estados compuestos

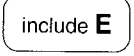
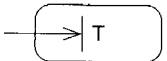
Un estado simple no tiene ninguna estructura, apenas tiene un conjunto de transiciones y posiblemente acciones de entrada y salida. Un estado compuesto es un estado que se ha descompuesto en subestados secuenciales o subestados concurrentes. La Tabla 6.4 enumera las distintas clases de estados.

Una descomposición en subestados disjuntos es un tipo de especialización de un estado.

Un estado externo se refina en varios estados internos, cada uno de los cuales hereda las transiciones del estado más externo. Únicamente puede ser activo a la vez un subestado secuencial. El estado externo representa la condición de estar en cualquiera de los estados internos.

Las transiciones hacia dentro o hacia fuera de un estado compuesto, invocan las acciones de entrada o de salida del estado. Si hay varios estados compuestos, una transición a través de varios niveles puede invocar múltiples acciones de entrada (primero las más exteriores) o de salida (primero las más internas).

Tabla 6.4 Tipos de Estado

<i>Tipo de estado</i>	<i>Descripción</i>	<i>Notación</i>
estado simple	Un estado sin estructura	
estado concurrente compuesto	Un estado que está dividido en dos o más estados concurrentes, todos los cuales están activos concurrentemente cuando el estado compuesto está activo	
estado secuencial compuesto	Un estado que contiene uno o más subestados disjuntos, exactamente uno de los cuales está activo en cada momento cuando el estado compuesto está activo	
estado inicial	Un pseudoestado que indica el estado inicial cuando es invocado el estado que lo engloba	
estado final	Un estado especial cuya activación indica que el estado que lo engloba ha completado su actividad	
estado de unión o conjunción	Un pseudoestado que encadena segmentos de transición en una sola transición atómica	
estado de historia	Un pseudoestado cuya activación restaura el estado activado previamente dentro de un estado compuesto	
estado de referencia a submáquina	Un estado que referencia a una submáquina, la cual está implícitamente insertada en el lugar del estado de referencia a submáquina	
estado abreviado externo	Un pseudoestado, dentro de un estado de referencia a submáquina, que identifica un estado en la referida	

Si hay una acción en la transición a sí mismo, la acción se ejecuta después de cualquier acción de salida y antes de que se ejecute cualquier acción de entrada.

Un estado compuesto también puede tener un estado inicial dentro de él. Una transición al límite del estado compuesto es implícitamente una transición al estado inicial. Un nuevo objeto comienza en su estado inicial de su estado más exterior. De la misma forma, un estado compuesto puede tener un estado final. Una transición al estado final acciona una transición de terminación (transición sin disparador) en el estado compuesto. Si un objeto alcanza el estado final de su estado más exterior, se destruye. Los estados iniciales, los estados finales, las acciones de entrada, y las acciones de salida permiten que la definición de un estado sea encapsulada independiente de las transiciones a y desde él.

La Figura 6.5 muestra una descomposición secuencial de un estado, incluyendo un estado inicial. Éste es el control para una máquina de venta de entradas.

Una descomposición en subestados concurrentes representa cómputos independientes. Cuando se entra en un superestado concurrente, el número de hilos de control aumenta. Cuando se

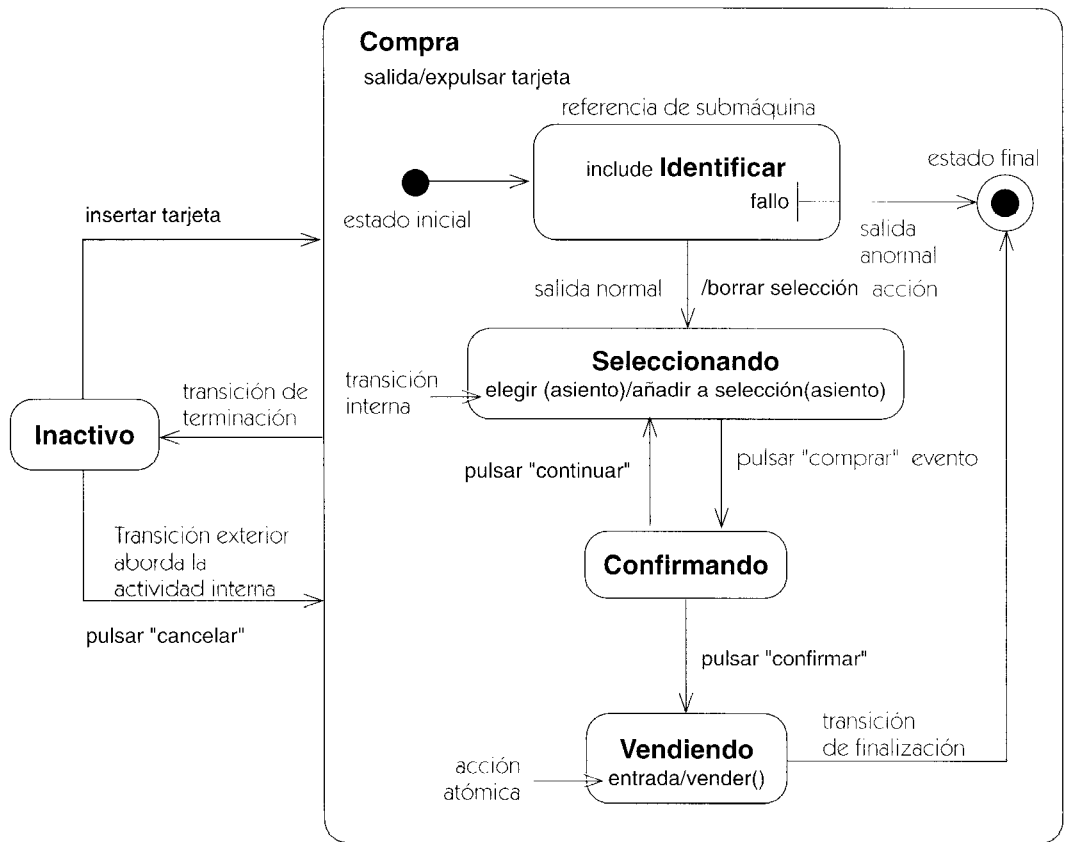


Figura 6.5 Máquina de estados

sale, el número de hilos de control disminuye. A menudo, la concurrencia es implementada por un objeto distinto para cada subestado, pero los subestados concurrentes pueden también representar concurrencia lógica dentro de un objeto. La Figura 6.6 muestra la descomposición concurrente de tomar una clase en la universidad.

A menudo, es conveniente reutilizar un fragmento de una máquina de estados en otras máquinas de estado. A una máquina de estados se le puede dar un nombre y hacer referencia a ella desde un estado de una o más máquinas. La máquina de estados de destino es una submáquina, y el estado que se refiere a él, se llama subestado de referencia a submáquina. Implica la sustitución (conceptual) de una copia de la máquina referida del estado en el lugar de la referencia, por una clase de subrutina de una máquina de estados. En vez de una submáquina, un estado puede contener una actividad, es decir, un cómputo o una ocurrencia continua que lleve un tiempo para terminarse y que pueda ser interrumpida por eventos. La Figura 6.7 muestra una referencia a submáquina.

Una transición a un estado de referencia a submáquina causa la activación del estado inicial de la submáquina destino. Para entrar en la submáquina en otros estados, se coloca uno o más estados externos en el estado de referencia a submáquina. Un estado externo identifica un estado en la submáquina.

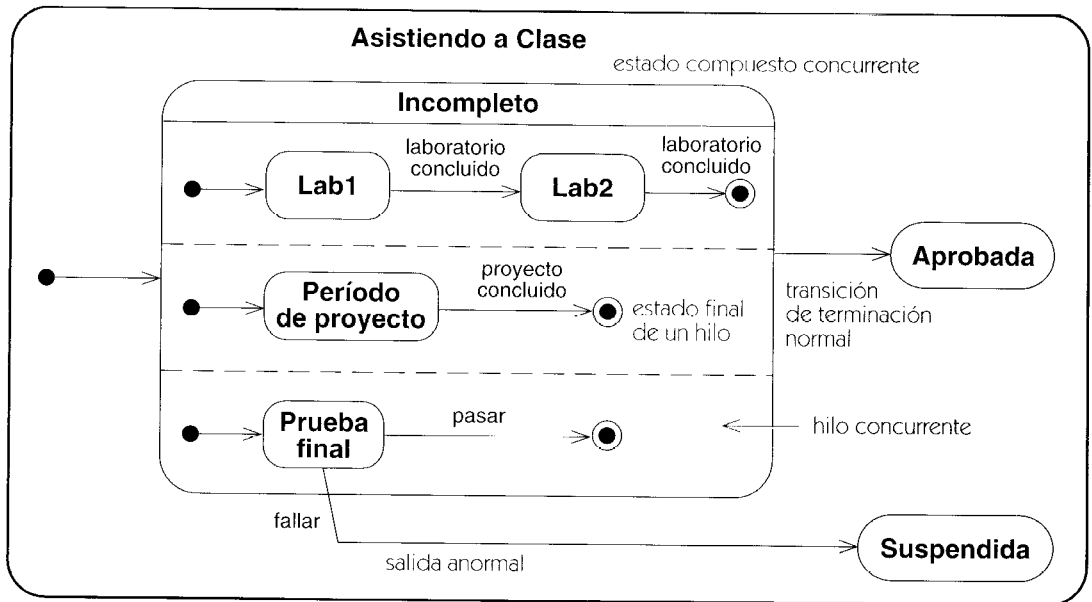
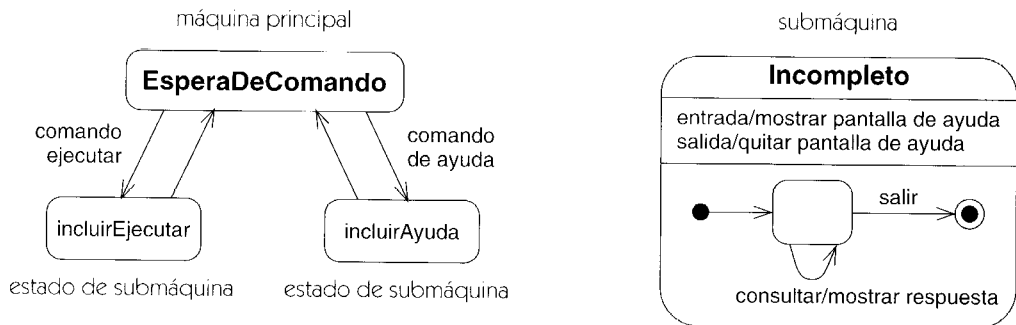


Figura 6.6 Máquina de estados con un estado compuesto concurrente



Esta submáquina puede ser utilizada muchas veces

Figura 6.7 Submáquina de estados

Descripción

Un grafo de actividades es una forma especial de máquina de estados, prevista para modelar cálculos y flujo de trabajos. Los estados del grafo de actividades representan los estados de ejecución del cálculo, no los estados de un objeto ordinario. Normalmente, un grafo de actividades asume que los cálculos proceden sin interrupciones externas por eventos (si las hubiera puede ser preferible una máquina de estados ordinaria).

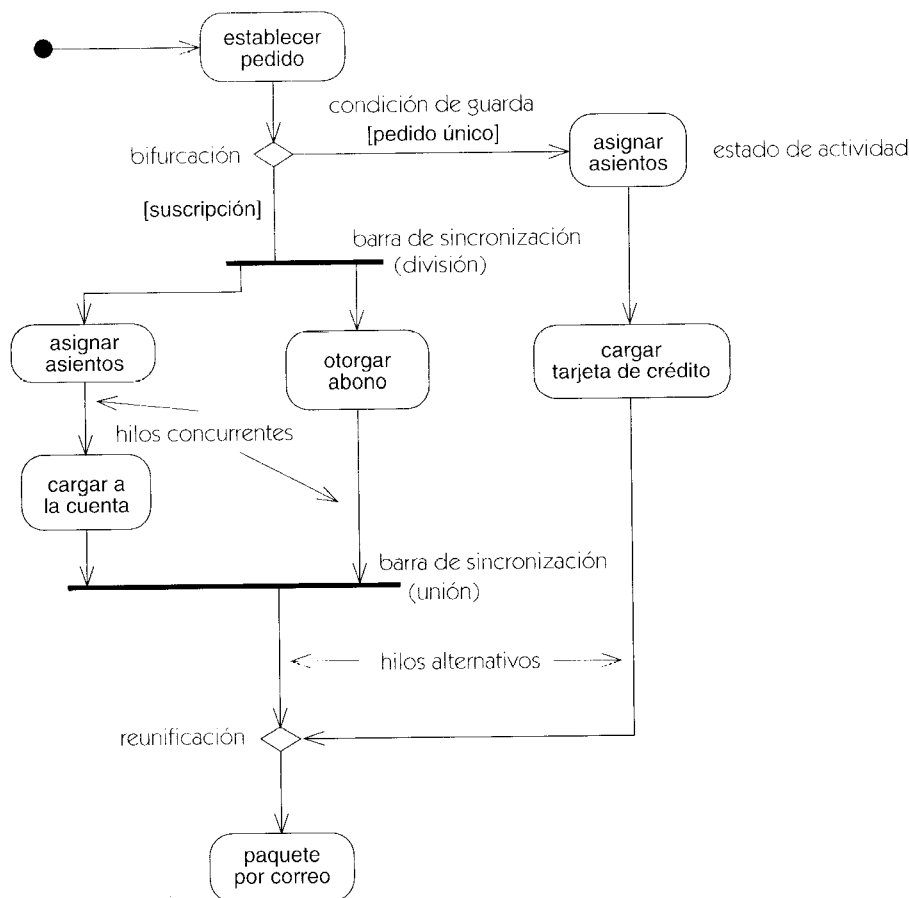
Un grafo de actividades contiene estados de actividad. Un estado de actividad representa la ejecución de una sentencia en un procedimiento, o el funcionamiento de una actividad en un flujo de trabajo. En vez de esperar un evento, como en un estado de espera normal, un estado de actividad espera la terminación de su cálculo. Cuando la actividad termina, entonces la ejecución procede al siguiente estado de actividad dentro del grafo. Una transición de terminación es activada en un diagrama de actividades cuando se completa la actividad precedente. Los estados de actividad no tienen generalmente transiciones con eventos explícitos, pero pueden ser abortados por transiciones en estados que los incluyen.

Un grafo de actividades puede también contener estados de acción, que son similares a los estados de actividad, excepto en que son atómicos y no permiten transiciones mientras están activos. Los estados de acción se deben utilizar generalmente para las operaciones cortas de mantenimiento.

Un diagrama de actividades puede contener bifurcaciones, así como divisiones de control en hilos concurrentes. Los hilos concurrentes representan actividades que se pueden realizar concurrentemente por los diversos objetos o personas en una organización. La concurrencia se presenta con frecuencia a partir de la agregación, en la cual cada objeto tiene su propio hilo concurrente. Las actividades concurrentes se pueden realizar simultáneamente o en cualquier orden. Un grafo de actividades es como un organigrama tradicional, excepto que permite control de concurrencia además de control secuencial: una gran diferencia.

Diagrama de actividades

Un diagrama de actividades es la notación para un grafo de actividades (Figura 7.1). Incluye algunos símbolos especiales abreviados por conveniencia. Estos símbolos pueden usarse en cualquier diagrama de estados, aunque mezclar la notación puede ser molesto.

Taquilla::ProcesarPedido**Figura 7.1** Diagrama de actividades

Un estado de actividad se representa como una caja con los extremos redondeados que contiene una descripción de actividad. (Las cajas normales del estado tienen lados rectos y esquinas redondeadas.) Las transiciones simples de terminación se muestran como flechas. Las ramas se muestran como condiciones de guarda en transiciones o como diamantes con múltiples flechas de salida etiquetadas. Una división o unión de control se representa de la misma manera que en un diagrama de estados, con múltiples flechas que entran o salen de una barra gruesa de sincronización. La Figura 7.1 muestra un diagrama de actividades para procesar un pedido por la taquilla.

Cuando es necesario incluir eventos externos, la recepción de un evento se puede mostrar como un disparador en una transición, o como un símbolo especial que denota la espera de una señal. Una notación similar muestra el envío de una señal. Sin embargo, si hay muchas transiciones dirigidas por eventos, probablemente es preferible un diagrama de estados ordinario.

Calles. A menudo es útil organizar las actividades en un modelo según su responsabilidad, por ejemplo, agrupando juntas todas las actividades manejadas por una organización del negocio. Esta clase de asignación puede mostrarse organizando las actividades en regiones dis-

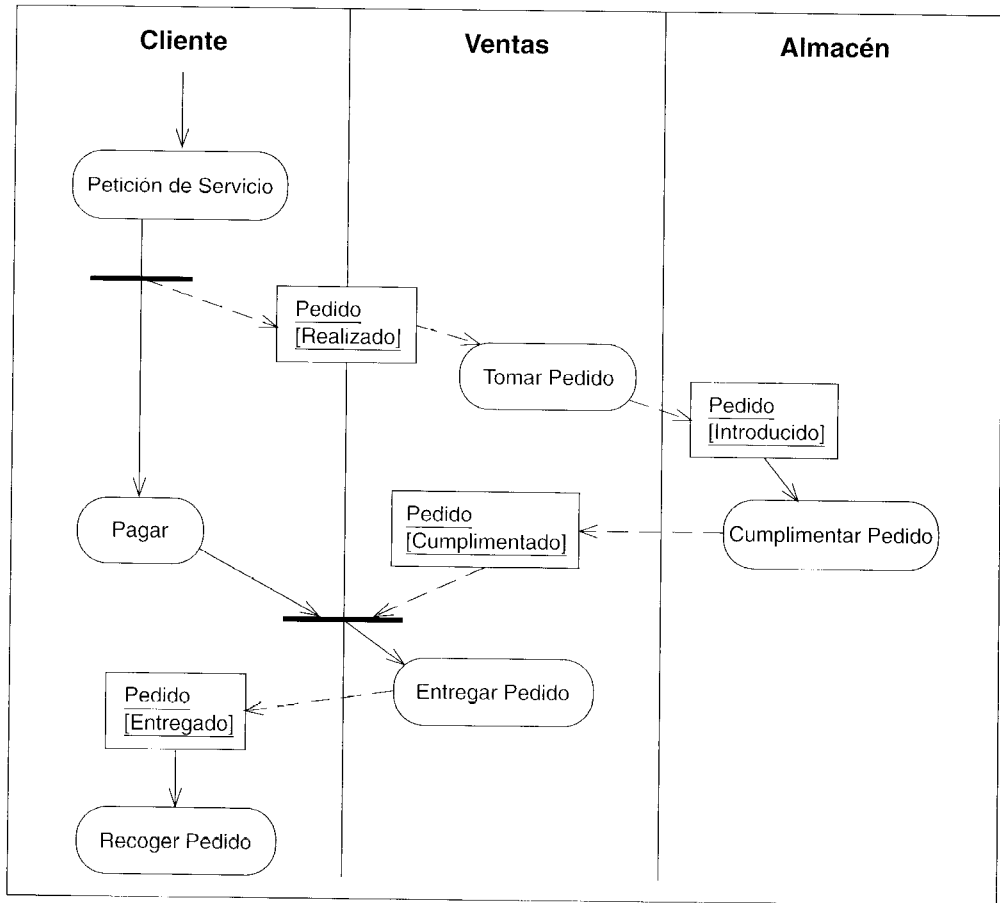


Figura 7.2 Calles y flujo de objetos

tintas separadas por líneas en el diagrama. Debido a su aspecto, cada región se llama calle¹. La Figura 7.2 muestra las calles.

Flujo de objetos. Un diagrama de actividades puede mostrar el flujo de objetos como valores, además del flujo de objeto. Se puede representar un objeto que sea la entrada o la salida de una actividad dibujando un objeto; también el estado del objeto, para indicar su evolución o flujo, dibujando un estado de flujo de objeto. Para un valor de salida, se dibuja una flecha con línea discontinua desde una actividad al objeto. Para un valor de entrada, se dibuja una flecha con línea discontinua desde el objeto a una actividad. Si una actividad tiene más de un flujo de salida o flujo de objeto sucesor, las flechas se dibujan desde el símbolo de división. De la misma forma, las entradas múltiples se dibujan hacia un símbolo de unión.

La Figura 7.2 muestra un diagrama de actividades en el cual las actividades y los estados de flujo de un objeto se han asignado a las calles.

¹ Por su aspecto y sentido, se utiliza el término “calle” como en algunas pistas deportivas; franjas que indican el desplazamiento de una persona concreta por ellas. (N. del Rev.)

Actividades y otras vistas

Los grafo de actividades no muestran el detalle completo de un cómputo. Muestran el flujo de actividades pero no los objetos que realizan las actividades. Los grafo de actividades son un punto de partida para el diseño. Para terminar un diseño, cada actividad se debe ampliar como una o más operaciones, cada una de las cuales para ser implementada se asigna a una clase específica. Tal asignación da lugar al diseño de una colaboración que implemente el grafo de actividades.

Descripción

Los objetos obran recíprocamente para implementar comportamiento. Esta interacción se puede describir de dos maneras complementarias, una de ellas se centra en los objetos individuales y la otra en una colección de objetos cooperantes.

Una máquina de estados es una vista estrecha y profunda del comportamiento, una vista reduccionista que mira a cada objeto individualmente. Una especificación de la máquina de estados es exacta y conduce inmediatamente al código. Sin embargo, puede ser difícil entender el funcionamiento total de un sistema, debido a que una máquina de estados se centra en un solo objeto a la vez, y se deben combinar los efectos de muchas máquinas de estado para determinar el comportamiento de todo el sistema. La vista de interacción proporciona una visión más integral del comportamiento de un sistema de objetos. Esta vista es modelada por colaboraciones.

Colaboración

Una colaboración es una descripción de una colección de objetos que interactúan para implementar un cierto comportamiento dentro de un contexto. Describe una sociedad de objetos cooperantes unidos para realizar un cierto propósito. Una colaboración contiene ranuras que son rellenadas por los objetos y enlaces en tiempo de ejecución. Una ranura de colaboración se llama rol, porque describe el propósito de un objeto o de un enlace, dentro de la colaboración. Un rol de clasificador representa una descripción de los objetos que pueden participar en una ejecución de la colaboración; un rol de asociación representa una descripción de los enlaces que pueden participar en una ejecución de la colaboración. Un rol de clasificador es un clasificador que está limitado por tomar parte en la colaboración; una rol de asociación es una asociación que está limitada por tomar parte en la colaboración.

Las relaciones entre roles de clasificador y roles de asociación dentro de una colaboración solamente son significativas en ese contexto. En general, no se aplican las mismas relaciones a los clasificadores y a las asociaciones subyacentes fuera de la colaboración.

La vista estática describe las propiedades inherentes de una clase. Por ejemplo, un **Vehículo** tiene un dueño. Una colaboración describe las propiedades que una instancia de una clase tiene porque desempeña un rol particular en una colaboración.

Por ejemplo, un **VehiculoAlquiler** en una colaboración **CocheAlquiler** tiene un **ConductorArrendatario**, algo que no es relevante a un **Vehículo** en general, pero es una parte esencial de la colaboración.

Un objeto en un sistema puede participar en más de una colaboración: Colaboraciones que parecen no estar relacionadas directamente, pero cuya ejecución está conectada a través de un objeto compartido. Por ejemplo, una persona puede ser un **ConductorArrendatario** y un **HuespedHotel** como parte de un modelo **Vacaciones**. Menos frecuentemente, un objeto puede desempeñar más de un rol en la misma colaboración.

Una colaboración tiene un aspecto estructural y un aspecto de comportamiento. El aspecto estructural es similar a una vista estática: contiene un conjunto de roles y de relaciones que definen el contexto para su comportamiento.

El comportamiento es el conjunto de mensajes intercambiados por los objetos ligados a los roles. Tal conjunto de mensajes en una colaboración se llama interacción. Una colaboración puede incluir una o más interacciones, cada una de las cuales describe una serie de mensajes intercambiados entre los objetos en la colaboración para realizar un objetivo.

Mientras que una máquina de estados es estrecha y profunda, una colaboración es amplia pero más superficial. Captura una vista más integral del comportamiento en el intercambio de mensajes dentro de una red de objetos. Las colaboraciones muestran la unidad de las tres estructuras importantes de computación subyacentes: la estructura de datos, el flujo de objeto, y flujo de datos.

Interacción

Una interacción es un conjunto de mensajes dentro de una colaboración que son intercambiados por roles de clasificador a través de roles de asociación. Cuando una colaboración existe en tiempo de ejecución, los objetos ligados a roles de clasificador intercambian instancias de mensajes a través de los enlaces ligados a los roles de asociación. Una interacción modela la ejecución de una operación, caso de uso, u otra entidad de comportamiento.

Un mensaje es una comunicación unidireccional entre dos objetos, un flujo de objeto con la información de un remitente a un receptor. Un mensaje puede tener parámetros que transporten valores entre los objetos. Un mensaje puede ser una señal (una comunicación explícita entre objetos, con nombre, y asíncrona) o una llamada (la invocación síncrona de una operación con un mecanismo para el control, que retorna posteriormente al remitente).

La creación de un nuevo objeto se modela como un evento causado por el objeto creador y recibido por la propia clase. El evento de creación está disponible para la nueva instancia como evento actual en la transición desde el estado inicial del objeto.

Los mensajes se pueden ordenar en hilos secuenciales de control. Los hilos separados representan conjuntos de mensajes concurrentes. La sincronización entre los hilos se modela mediante restricciones entre mensajes en diversos hilos. Una construcción de sincronización puede modelar divisiones del control, uniones del control, y bifurcaciones.

El orden de los mensajes se puede mostrar en dos tipos de diagramas: un diagrama de secuencia (que se centra en las secuencias de tiempo de los mensajes) y un diagrama de colaboración (que se centra en las relaciones entre los objetos que intercambian los mensajes).

Diagrama de secuencia

Un diagrama de secuencia representa una interacción como un gráfico bidimensional. La dimensión vertical es el eje de tiempo, que avanza hacia abajo de la página. La dimensión horizon-

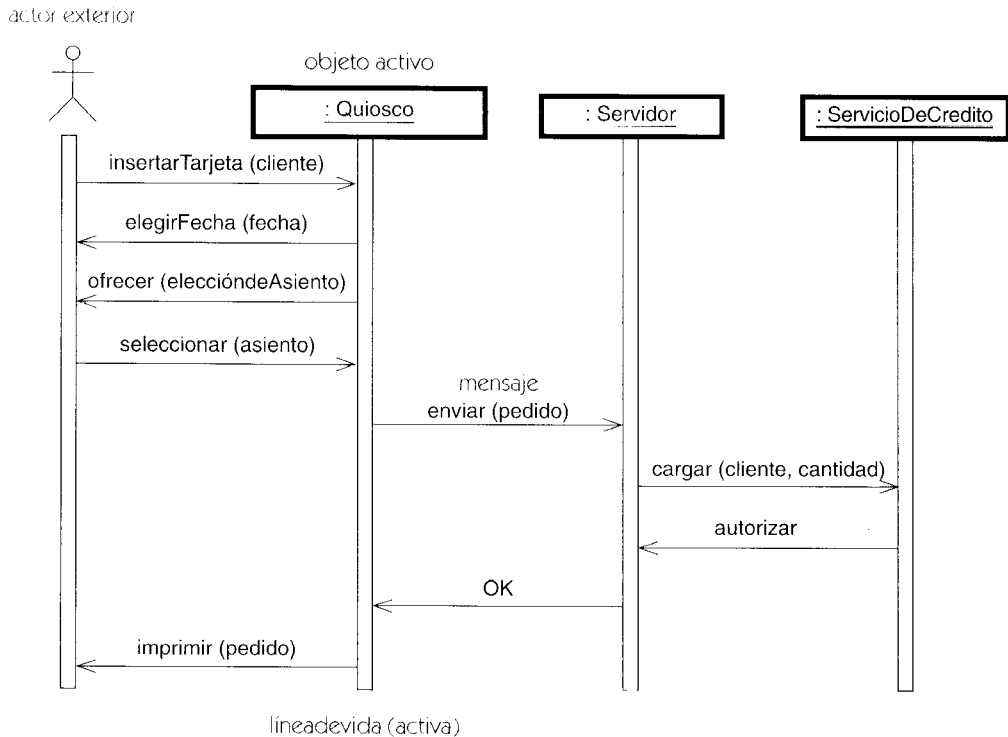


Figura 8.1 Diagrama de secuencia

tal muestra los roles de clasificador que representan objetos individuales en la colaboración. Cada rol de clasificador se representa mediante una columna vertical-línea de vida. Durante el tiempo que existe un objeto, el rol se muestra por una línea discontinua. Durante el tiempo que dura una activación de un procedimiento en el objeto, la línea de vida se dibuja como una línea doble.

Se muestra un mensaje como una flecha desde la línea de vida de un objeto a la del otro. Las flechas se organizan en el diagrama en orden cronológico hacia abajo.

La Figura 8.1 muestra un diagrama de secuencia típico con mensajes asíncronos.

Activación

Una activación es la ejecución de un procedimiento, incluyendo el tiempo que espera a los procedimientos anidados para ejecutarse. Se representa por una línea doble que sustituye la parte de la línea de vida en un diagrama de secuencia. Una llamada se representa por una flecha que apunta a la parte superior de la activación iniciada por la llamada. Ocurre una llamada recursiva cuando el control vuelve a entrar en una operación en un objeto, pero la segunda llamada es una activación separada de la primera. La recursión o una llamada anidada a otra operación en el mismo objeto, se representa en un diagrama de secuencia apilando las líneas de activación. La Figura 8.2 muestra un diagrama de secuencia, con el flujo de objeto de procedimientos, incluyendo una llamada recursiva y la creación de un objeto durante el cómputo.

Un objeto activo es un objeto que contiene la raíz de una pila de activaciones. Cada objeto activo tiene su propio hilo de control dirigido por eventos que se ejecuta en paralelo a otros ob-

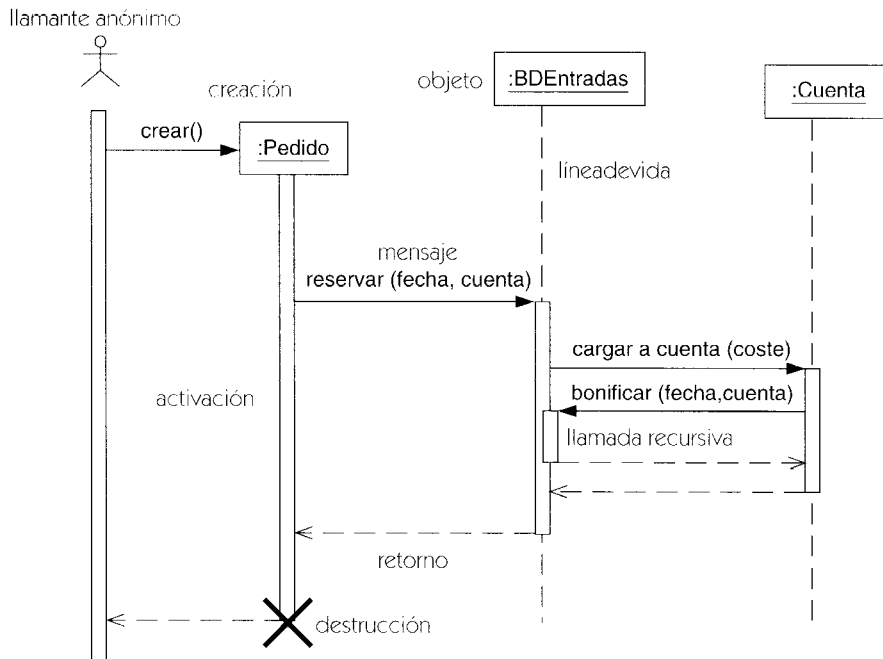


Figura 8.2 Cuando dos hilos van juntos en un solo objeto

jetos activos. Los objetos que son llamados por un objeto activo son objetos pasivos; reciben el control solamente cuando son llamados, y lo ceden cuando retornan.

Si varios hilos de control concurrentes tienen sus propios flujos de procedimientos de control usando llamadas anidadas, los diversos hilos deben distinguirse usando los nombres de los hilos, colores, u otros medios para evitar la confusión cuando dos hilos se unen en un único objeto (para una sincronización, por ejemplo). Generalmente, es mejor no mezclar llamadas a procedimientos con señales en un solo diagrama.

Diagrama de colaboración

Un diagrama de colaboración es un diagrama de clases que contiene roles de clasificador y roles de asociación en lugar de sólo clasificadores y asociaciones. Los roles de clasificador y los roles de asociación describen la configuración de los objetos y de los enlaces que pueden ocurrir cuando se ejecuta una instancia de la colaboración. Cuando se instancia la colaboración, los objetos están ligados a los roles de clasificador y los enlaces están ligados a los roles de asociación. El rol de asociación también puede ser desempeñado por varios tipos de enlaces temporales, tales como argumentos de procedimiento o variables locales del procedimiento. Los símbolos de enlace pueden llevar estereotipos para indicar enlaces temporales («**parameter**» o «**local**») o llamadas al mismo objeto («**self**»).

Solamente se representan los objetos que están implicados en la colaboración, aunque puede haber otros en el sistema completo. Es decir, un diagrama de colaboración modela los objetos y los enlaces implicados en la implementación de una interacción y no hace caso de las otras. La Figura 8.3 muestra un diagrama de colaboración.

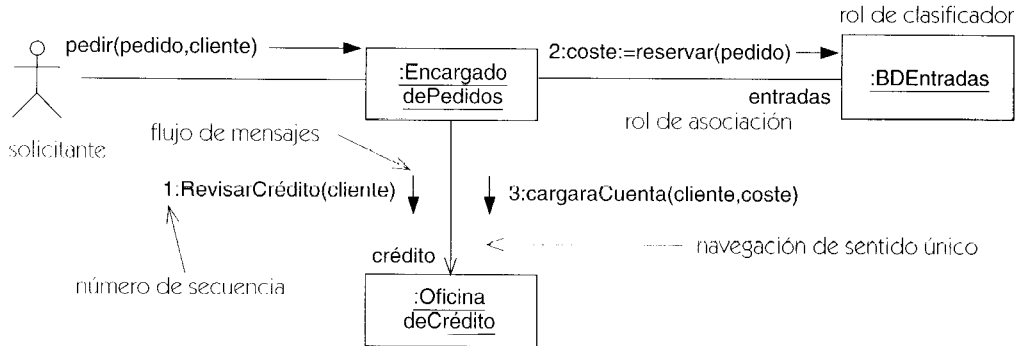


Figura 8.3 Diagrama de colaboración

Es útil marcar los objetos en cuatro grupos: los que existen con la interacción entera; los creados durante la interacción (restricción **{new}**); los destruidos durante la interacción (restricción **{destroyed}**); y los que se crean y se destruyen durante la interacción (restricción **{transient}**). Durante el diseño, se puede empezar mostrando los objetos y los enlaces disponibles al comienzo de una operación y después decidir cómo el control puede fluir a los objetos correctos dentro del gráfico para implementar la operación.

Aunque las colaboraciones muestran directamente la implementación de una operación, pueden también mostrar la realización de una clase entera. En este uso, muestran el contexto necesario para implementar *todas* las operaciones de una clase. Esto permite que el modelador vea los roles múltiples que los objetos pueden desempeñar en varias operaciones. Esta vista puede ser construida tomando la unión de todas las colaboraciones necesarias para describir todas las operaciones del objeto.

Mensajes. Los mensajes se muestran como flechas etiquetadas unidas a los enlaces. Cada mensaje tiene un número de secuencia, una lista opcional de mensajes precedentes, una condición opcional de guarda, un nombre y lista de argumentos, y un nombre de valor de retorno opcional. El número de serie incluye el nombre (opcional) de un hilo. Todos los mensajes del mismo hilo se ordenan secuencialmente. Los mensajes de diversos hilos son concurrentes a menos que haya una dependencia secuencial explícita. Pueden añadirse varios detalles de implementación, como por ejemplo una distinción entre los mensajes síncronos y asíncronos.

Flujos. Generalmente, un diagrama de colaboración contiene un símbolo para un objeto durante una operación completa. Sin embargo, a veces, un objeto tiene distintos estados que se deban hacer explícitos. Por ejemplo, un objeto pudo cambiar de localización, o sus asociaciones pudieran diferenciarse perceptiblemente en distintos momentos. Un objeto se puede mostrar con su clase y su estado —un objeto de una clase en un estado—. El mismo objeto se puede mostrar múltiples veces, cada uno con una localización o estado diferentes.

Los diferentes símbolos de objeto que representan un objeto se pueden conectar usando flujos «**become**» o «**conversión**». Un flujo «**become**» es una transición, a partir de un estado de un objeto a otro. Se dibuja como una flecha de línea discontinua con el estereotipo «**become**» o «**conversión**» y puede ser etiquetado con un número de serie para mostrar cuando ocurre (Figura 8.4). Un flujo de conversión también se utiliza para mostrar la migración de un objeto a partir de una localización a otra distinta.

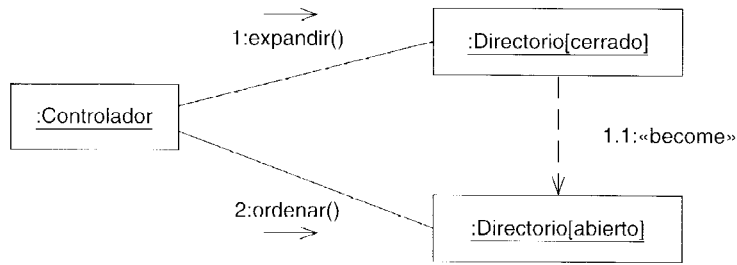


Figura 8.4 Flujos de conversión

Menos frecuentemente, el estereotipo «copy» o «copia» muestra un valor de objeto producido copiando otro valor de objeto.

La Tabla 8.1 muestra los tipos de relaciones de flujo de objeto.

Diagramas de colaboración y de secuencia. Los diagramas de colaboración y los diagramas de secuencia muestran interacciones, pero acentúan aspectos diferentes. Los diagramas de secuencia muestran secuencias de tiempo claramente pero no muestran explícitamente relaciones de objetos. Los diagramas de colaboración muestran las relaciones entre objetos claramente, pero las secuencias de tiempo se deben obtener a partir de números de secuencia. Los diagramas de secuencia son a menudo más útiles para mostrar escenarios; los diagramas de colaboración son a menudo más útiles para mostrar el diseño detallado de procedimientos.

Tabla 8.1 Tipos de Relaciones de Flujo

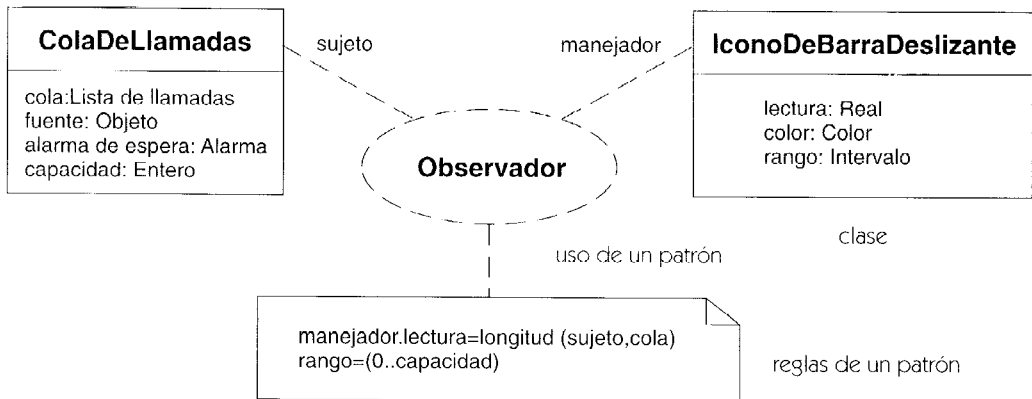
Flujo	Función	Notación
conversión	Transformación de un valor de un objeto a otro valor	«become» ----->
copia	Copia de un objeto que a partir de entonces es independiente	«copy» ----->

Patrones

Un patrón es una colaboración parametrizada, junto con las pautas sobre cuándo utilizarlo. Un parámetro se puede sustituir por diversos valores, para producir distintas colaboraciones. Los parámetros señalan generalmente las ranuras para las clases. Cuando se instancia un patrón, sus parámetros están ligados a clases reales de un diagrama de clase o a los roles dentro de una colaboración más amplia.

El uso de un patrón se representa como una elipse de línea discontinua conectada con cada una de sus clases por una línea discontinua, que se etiqueta con el nombre del rol. Por ejemplo, la Figura 8.5 muestra el uso del patrón **Observer** de [Gamma-95]. En este uso del patrón, **ColadeLlamadas** sustituye el rol del **sujeto** e **IconoDeBarraDeslizante** sustituye el rol del **manejador**.

Los patrones pueden aparecer en los niveles de análisis, arquitectura, diseño detallado e implementación. Son una manera de captar estructuras que ocurren frecuentemente para su reutilización. La Figura 8.5 muestra un uso del patrón Observer (Observador).

**Figura 8.5** Uso del patrón

Descripción

Gran parte de un modelo de un sistema se piensa para mostrar los aspectos lógicos y de diseño del sistema, independiente de su empaquetado final en un medio de implementación. Sin embargo, los aspectos de implementación son importantes para los propósitos de reutilización y del rendimiento. UML incluye dos tipos de vistas para representar unidades de implementación: la vista de implementación y la vista de despliegue.

La vista de implementación muestra el empaquetado físico de las partes reutilizables del sistema en unidades sustituibles, llamadas componentes. Una vista de implementación muestra la implementación de los elementos del diseño (tales como clases) mediante componentes, así como sus interfaces y dependencias entre componentes. Los componentes son las piezas reutilizables de alto nivel a partir de las cuales se pueden construir los sistemas.

La vista de despliegue muestra la disposición física de los recursos de ejecución computacional, tales como computadores y sus interconexiones. Se llaman nodos. Durante la ejecución, los nodos pueden contener componentes y objetos. La asignación de componentes y de objetos a los nodos puede ser estática, o pueden migrar entre nodos.

La vista de despliegue puede mostrar cuellos de botella para el rendimiento si las instancias de los componentes con dependencias se ponen en distintos nodos.

Componente

Un componente es una unidad física de implementación con interfaces bien definidas pensada para ser utilizada como parte reemplazable de un sistema. Cada componente incorpora la implementación de ciertas clases del diseño del sistema. Los componentes bien diseñados no dependen directamente de otros componentes sino de las interfaces que ofrecen los componentes. En ese caso, un componente en un sistema se puede sustituir por otro componente que ofrezca las interfaces apropiadas.

Los componentes soportan más interfaces y requieren ciertas interfaces de otros componentes. Una interfaz es una lista de las operaciones que una pieza de software o de hardware ofrece y puede realizar. El uso de las llamadas interfaces permite evitar las dependencias directas entre componentes, facilitando una sustitución más fácil de nuevos componentes. La vista de componentes muestra la red de dependencias entre componentes. La vista de componentes puede aparecer de dos formas. Puede mostrar un conjunto de componentes disponibles (una biblioteca de componentes) con sus dependencias; éste es el material a partir del cual se puede ensam-

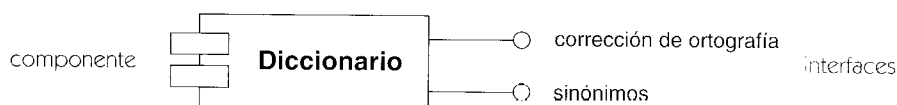


Figura 9.1 Componente con interfaces

blar un sistema. Puede también mostrar un sistema configurado, con la selección de componentes usados para construirlo (a partir de la biblioteca completa). De esta forma, cada componente se conecta a otros componentes cuyos servicios utiliza; estas conexiones deben ser consistentes con las interfaces de los componentes.

Un componente se dibuja como un rectángulo, con dos rectángulos pequeños a un lado. Puede ser unido por líneas sólidas a los círculos que representan sus interfaces (Figura 9.1).

Un diagrama de componentes muestra dependencias entre los componentes (Figura 9.2). Cada componente ofrece algunas interfaces y utiliza otras. Si las dependencias entre componentes se hacen a través de interfaces, los componentes se pueden sustituir por otros componentes que realicen las mismas interfaces.

Nodo

Un nodo es un objeto físico de ejecución que representa un recurso computacional, que generalmente tiene por lo menos memoria y a menudo también capacidad de proceso. Los nodos pueden tener estereotipos para distinguir diferentes tipos de recursos, tales como UCP, dispositivos, y memorias. Los nodos pueden contener objetos, instancias, instancias del componente.

Un nodo se representa mediante un cubo estilizado con el nombre del nodo y, opcionalmente, su clasificación (Figura 9.3).

Las asociaciones entre los nodos representan líneas de comunicación. Las asociaciones pueden tener estereotipos para distinguir diversos tipos de enlaces.

Los nodos pueden tener relaciones de generalización para relacionar una descripción general de un nodo con una variación más específica.

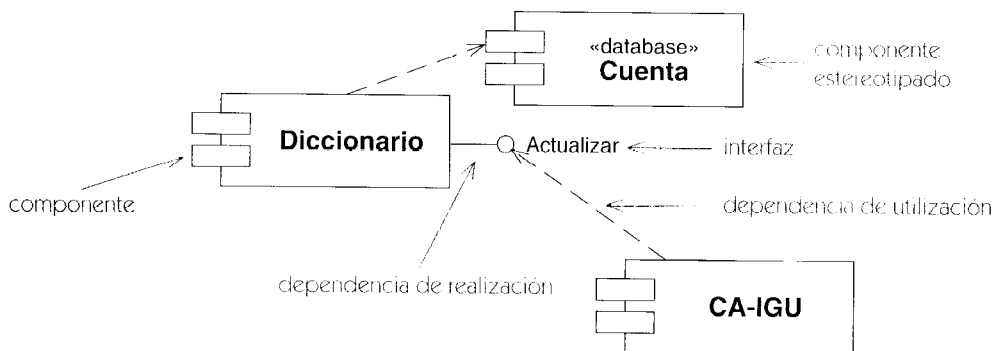


Figura 9.2 Diagrama de componentes

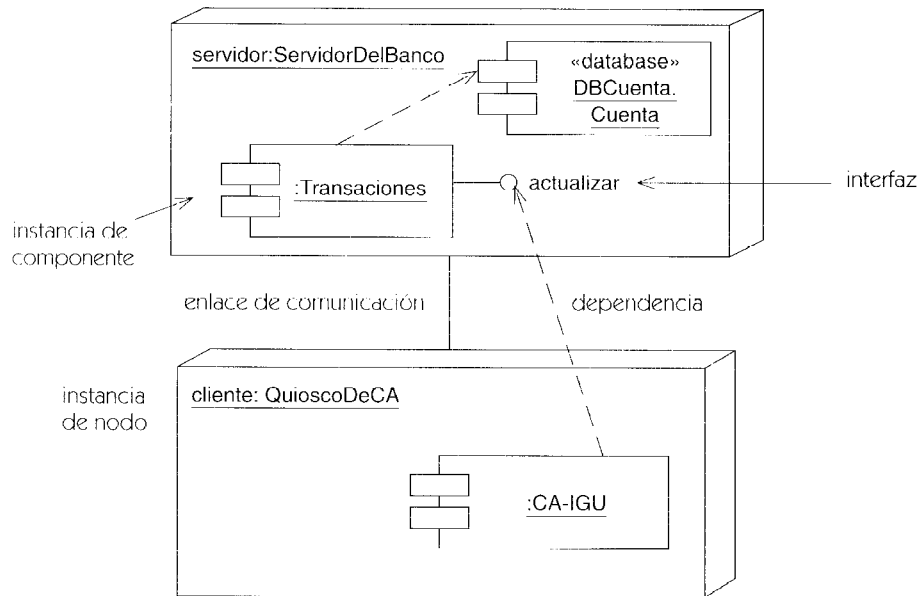


Figura 9.3 Diagrama de despliegue

La presencia de un objeto en un nodo se representa mediante el anidamiento físico del símbolo de objeto dentro del símbolo del nodo. Si eso no es conveniente, el símbolo de objeto puede contener la etiqueta **location**, cuyo valor sea el nombre del nodo en el que reside el objeto (su localización). La migración de objetos o de instancias de componentes entre nodos, puede representarse también.

Véase conversión.

Descripción

Cualquier sistema grande se debe dividir en unidades más pequeñas, de modo que las personas puedan trabajar con una cantidad de información limitada, a la vez y de modo que los equipos de trabajo no interfieran con el trabajo de los otros. La gestión del modelo consiste en paquetes (incluyendo tipos especiales de paquetes) y relaciones de dependencia entre paquetes.

Paquete

Un paquete es una parte de un modelo. Cada parte de un modelo debe pertenecer a un paquete. El modelador puede asignar el contenido de un modelo a un conjunto de paquetes. Pero, para ser funcional, la asignación debe seguir un cierto principio racional, tal como funcionalidad común, implementación estrechamente relacionada, y un punto de vista común. UML no impone una regla para componer los paquetes, pero una buena descomposición en paquetes realizará enormemente la capacidad de mantenimiento del modelo.

Los paquetes contienen elementos del modelo al más alto nivel, tales como clases y sus relaciones, máquinas de estado, diagramas de casos de uso, interacciones, y colaboraciones: cualquier elemento que no esté contenido en otro. Los elementos como atributos, operaciones, estados, líneas de vida, y mensajes están contenidos en otros elementos, y no aparecen como contenido directo de los paquetes. Cada elemento de nivel superior tiene un paquete en el cual se declara. Éste es su paquete de origen. Puede hacerse referencia a él en otros paquetes, pero el contenido del elemento pertenece al paquete de origen. En un sistema de control de configuración, un modelador debe tener acceso al paquete de origen para modificar el contenido de un elemento. Esto proporciona un mecanismo de control de acceso para trabajar en los modelos grandes. Los paquetes son también las unidades para cualquier mecanismo de versiones.

Los paquetes pueden contener otros paquetes. Hay un paquete raíz, que contiene indirectamente el modelo completo de un sistema. Hay varias maneras posibles de organizar los paquetes en un sistema. Pueden ser organizados por la vista, por la funcionalidad, o por cualquier otra base que el modelador elija. Los paquetes son unidades de organización jerárquica de uso general de los modelos de UML. Pueden ser utilizados para el almacenamiento, el control de acceso, la gestión de la configuración, y la construcción de bibliotecas que contengan fragmentos reutilizables del modelo.

Si se eligen bien los paquetes, reflejan la arquitectura de alto nivel de un sistema: su descomposición en subsistemas y sus dependencias. Una dependencia entre paquetes resume las dependencias entre los contenidos del paquete.

Dependencias en los paquetes

Las dependencias que se presentan entre elementos individuales, pero en un sistema de cualquier tamaño, deben ser vistas en un nivel más alto. Las dependencias entre los paquetes resumen dependencias entre los elementos internos a ellos, es decir, las dependencias del paquete son derivables a partir de las dependencias entre elementos individuales.

La presencia de una dependencia entre paquetes implica que existe en un enfoque ascendente (una declaración de existencia), o que se permite que exista más adelante en un enfoque descendente (una restricción que limita cualquier otra relación), por lo menos un elemento de relación con el tipo indicado de dependencia entre elementos individuales dentro de los paquetes correspondientes. Es una declaración de existencia y no implica que todos los elementos del paquete tengan la dependencia. Es un indicador para el modelador de que existe información adicional, pero la dependencia, cuando se trata de paquetes, no contiene más información por sí misma; es solamente un resumen.

El enfoque descendente refleja la arquitectura del sistema total. El enfoque ascendente se puede generar automáticamente a partir de los elementos individuales. Ambos enfoques tienen su lugar en el modelado, incluso en un solo sistema.

Las dependencias múltiples del mismo tipo entre elementos individuales se agregan a una sola dependencia entre los paquetes que contienen los elementos. Si las dependencias entre elementos individuales contienen estereotipos (tales como diferentes tipos de uso), el estereotipo se puede omitir en la dependencia de paquete, para dar una sola dependencia de alto nivel.

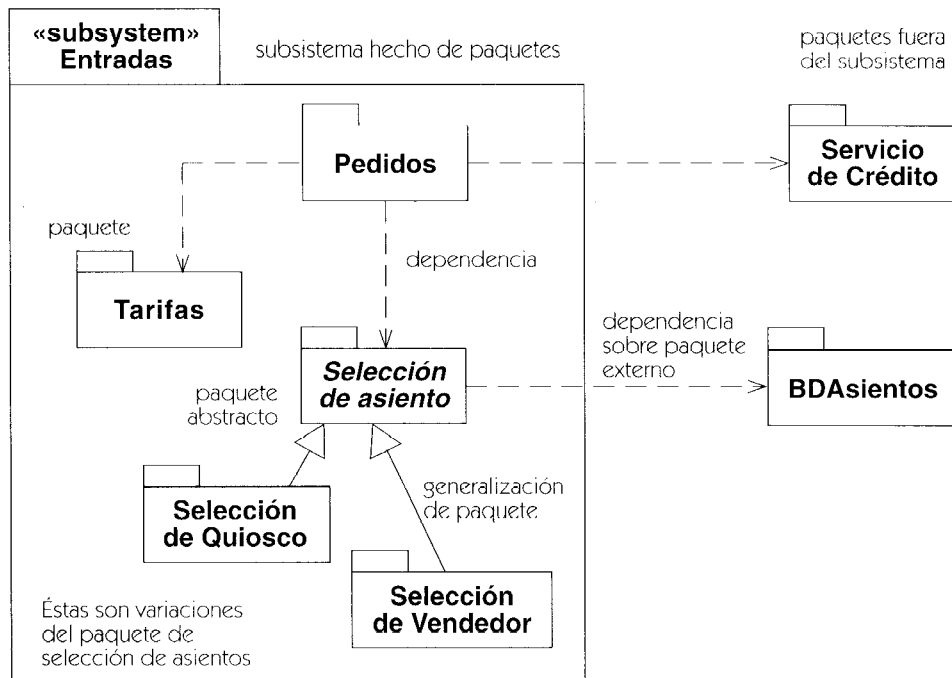


Figura 10.1 Paquetes y sus relaciones

Los paquetes se dibujan como rectángulos con lengüetas (iconos “carpeta”). Las dependencias se muestran como flechas con líneas discontinuas.

La Figura 10.1 muestra la estructura del paquete para un subsistema de reserva de entradas. Tiene dependencias en los paquetes exteriores y dos variaciones del paquete de **selección de asientos**. Cualquier implementación del subsistema incluiría solamente una variación.

Dependencia de acceso e importación

En general, un paquete no puede tener acceso al contenido de otro paquete. Los paquetes son opacos, a menos que sean abiertos por una dependencia de acceso o de importación. La dependencia de acceso se aplica directamente a los paquetes y a otros contenedores. En el nivel de paquete, la dependencia de acceso indica que el contenido del paquete del proveedor puede aparecer en referencias efectuadas por los elementos del paquete cliente o de los paquetes incluidos dentro del cliente. Un elemento en el proveedor debe tener suficiente visibilidad dentro de su paquete para permitir a un cliente verlo. En general, un paquete puede ver solamente los elementos de otros paquetes que tienen asignada visibilidad pública en el paquete que los contiene. Los elementos con visibilidad protegida son visibles solamente por los paquetes que son descendientes del paquete que contiene los elementos. Los elementos con visibilidad privada son visibles solamente por el paquete que los contiene, y por cualesquiera paquetes anidados en el interior de ese paquete. La visibilidad también se aplica al contenido de clases (atributos y operaciones). Un descendiente de una clase puede ver a miembros de su antecesor con visibilidad pública o protegida; cualquier otra clase puede ver solamente miembros con visibilidad pública. El permiso de acceso y la visibilidad apropiada son necesarios para hacer referencia a un elemento. Así que para que un elemento, en un paquete, vea a un elemento en un paquete sin relación, el primer paquete debe tener acceso o importar al segundo paquete, y el elemento destino debe tener visibilidad pública dentro del segundo paquete.

Un paquete anidado dentro de otro paquete es parte del contenedor y tiene acceso completo a su contenido, sin necesidad de accesos. El contenedor, sin embargo, puede no ver el interior de sus paquetes anidados si no tiene acceso. Se encapsula el contenido.

Observe que una dependencia de acceso no modifica el espacio de nombres del cliente ni crea las referencias automáticamente de ninguna otra forma. Simplemente concede permiso para establecer referencias. La dependencia de importación se utiliza para agregar nombres al espacio de nombres del paquete del cliente como sinónimos de los caminos completos.

Modelo y subsistema

Un modelo es un paquete que abarca una descripción completa de una vista particular de un sistema. Proporciona una descripción cerrada de un sistema a partir de un punto de vista. No tiene dependencias fuertes en otros paquetes, tales como dependencias de implementación o dependencias de herencia. La relación de traza es una forma débil de dependencia entre elementos, en distintos modelos, que observan la presencia de una cierta conexión sin implicaciones semánticas específicas.

Generalmente, un modelo se estructura con forma de árbol. El paquete raíz contiene anidados en sí mismo paquetes que constituyen el detalle completo del sistema desde un punto de vista dado.

Un subsistema es un paquete que tiene piezas separadas de especificación y de realización. Representa una unidad coherente del modelo, con interfaces limpias al resto del sistema. Representa generalmente la partición del sistema, en un límite funcional o de implementación. Los modelos y los subsistemas se dibujan como paquetes con las palabras clave de estereotipo (Figura 10.1).

Descripción

UML proporciona varios mecanismos de extensión para permitir que los modeladores hagan algunas extensiones comunes sin tener que modificar el lenguaje de modelado subyacente. Se han diseñado estos mecanismos de extensión de modo que las herramientas puedan almacenar y manipular las extensiones aun sin entender su semántica completa. Por esta razón, las extensiones se pueden almacenar y manipular como cadenas. Para una herramienta que no entienda la extensión, es sólo una cadena, pero se puede introducir o almacenar como parte de un modelo, y ser pasada a otras herramientas. Se espera que las herramientas de base y los módulos adicionales sean escritos para procesar varios tipos de extensiones. Estas herramientas definirán una sintaxis y una semántica particulares para sus extensiones que sólo ellas necesitan entender.

Este acercamiento a las extensiones no resolverá probablemente todas las necesidades que se presenten, pero pensamos que se ajustará a una porción grande de las adaptaciones que necesitan la mayoría de los modeladores de una manera simple que es fácil de implementar. Los mecanismos de extensión son restricciones, valores etiquetados, y estereotipos. Tenga presente que una extensión, por definición, desvía la forma de estándar de UML y puede, por lo tanto, conducir a problemas de interoperatividad. El modelador debe sopesar cuidadosamente ventajas y costes antes de usar extensiones, especialmente cuando los mecanismos existentes trabajaran razonablemente bien. Típicamente, las extensiones están pensadas para los dominios de aplicaciones particulares o los entornos de programación, pero dan lugar a un dialecto de UML, con las ventajas y desventajas de todos los dialectos.

Restricción

Una restricción es una condición semántica representada como expresión textual. Cada expresión tiene un lenguaje implícito de interpretación, que puede ser una notación matemática formal, tal como notación de teoría de conjuntos; un lenguaje computarizado de restricción, tal como OCL; un lenguaje de programación, como C++; o pseudocódigo o el lenguaje natural informal. Por supuesto, si el lenguaje es informal, entonces su interpretación es también informal, y debe ser hecha por una persona. Incluso si una restricción se expresa en un lenguaje formal, no significa que se hará cumplir automáticamente. Las técnicas de mantenimiento y comprobación automática de verdades no pueden cubrir la mayoría de los casos, pero por lo menos la semántica será exacta.

Las restricciones pueden expresar limitaciones y relaciones que no se pueden expresar usando la notación de UML. Son particularmente útiles para indicar las condiciones globales o las condiciones que afectan a ciertos elementos.

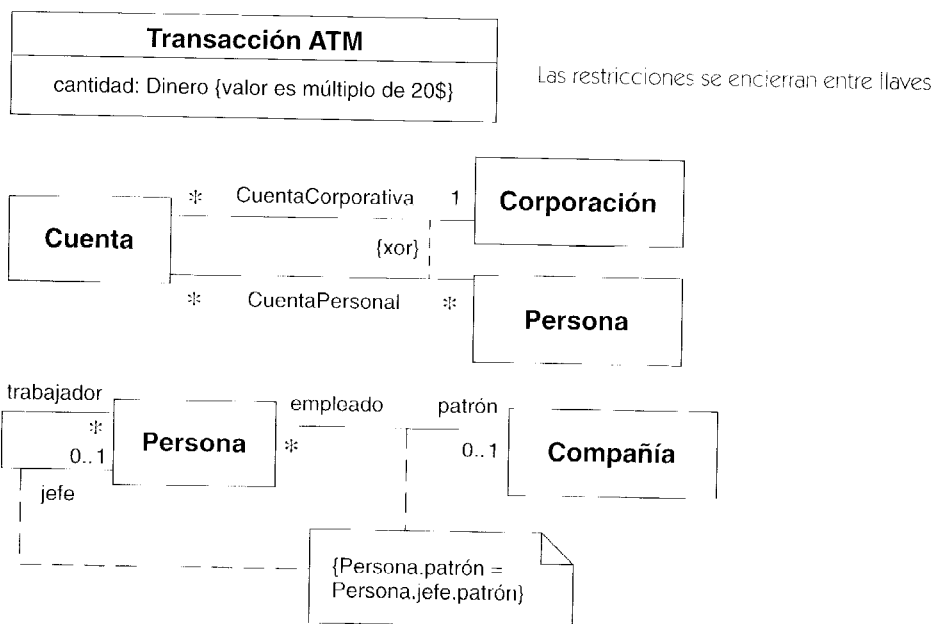


Figura 11.1 Restricciones

Las restricciones se muestran como expresiones de cadena entre llaves. Pueden ser añadidas a un elemento de una lista, ser asociadas a una dependencia, o ser incluidas en una nota. La Figura 11.1 muestra varios tipos de restricciones.

Valor etiquetado

Un valor etiquetado es un par de cadenas: una cadena de etiqueta y una cadena de valor, que almacena un elemento de información sobre un elemento. Un valor etiquetado se puede unir a cualquier elemento individual, incluyendo los elementos del modelo y los elementos de la presentación.

La etiqueta es un nombre de una cierta propiedad que el modelador desea registrar, y el valor es el valor de esa propiedad para el elemento dado. Por ejemplo, la etiqueta pudo ser **autor**, y el valor pudo ser el nombre de la persona responsable del elemento, tal como **Charles Babbage**.

Los valores etiquetados se pueden utilizar para almacenar cualquier información sobre los elementos. Son particularmente útiles para almacenar información de gestión del proyecto, tal como la fecha de creación de un elemento, de su estado de desarrollo, de fechas debidas, y del estado de la prueba. Cualquier cadena se puede utilizar como un nombre de etiqueta, excepto aquellos nombres de atributos incorporados al metamodelo que deben evitarse (porque las etiquetas y los atributos juntos pueden considerarse propiedades de un elemento y accederse de manera uniforme en una herramienta), y ciertos nombres de etiqueta están predefinidos (véase el capítulo 14, Elementos Estándar).

Los valores etiquetados también proporcionan una manera de añadir información dependiente de la implementación a los elementos. Por ejemplo, un generador de código necesita la información adicional sobre el tipo de código a generar a partir de un modelo. A menudo, hay

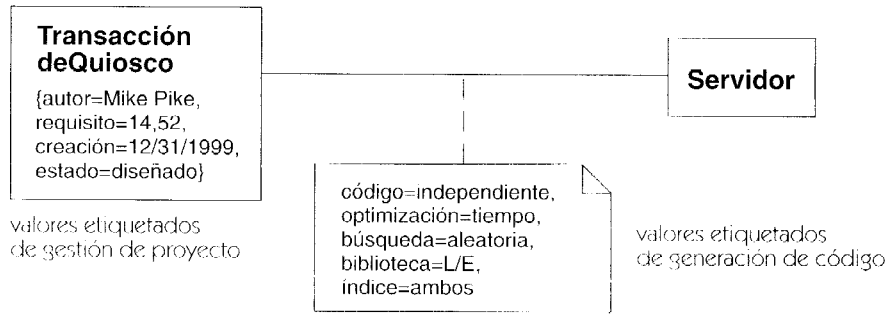


Figura 11.2 Valores etiquetados

varias maneras posibles de implementar correctamente un modelo; el modelador debe proporcionar las directrices sobre qué elección tomar. Ciertas etiquetas se pueden utilizar como indicadores para decir al generador de código qué implementación utilizar. Otras etiquetas se pueden utilizar para otros tipos de herramientas añadidas, tales como planificadores del proyecto y generadores de informes.

Los valores etiquetados también se pueden utilizar para almacenar información sobre elementos estereotipados del modelo (discutidos abajo).

Los valores etiquetados se muestran como cadenas con el nombre de la etiqueta, un igual, y el valor. Se colocan normalmente en listas entre llaves (Figura 11.2). Serán omitidos en los diagramas pero mostrados a menudo en listas y formularios emergentes.

Estereotipos

Muchos modeladores desean adaptar un lenguaje de modelado para un dominio de aplicación particular. Esto conlleva un cierto riesgo, porque el lenguaje adaptado no será universalmente comprensible, pero existe, sin embargo, la tentación de hacerlo.

Un estereotipo es un tipo de elemento del modelo definido en el propio modelo. El contenido de información y la forma de un estereotipo son igual que los de una clase existente de elemento del modelo base, pero su significado y uso son diferentes. Por ejemplo, los modeladores en el área de modelado de negocio a menudo quieren distinguir objetos de negocio y procesos del negocio como tipos especiales de elementos de modelado cuyo uso es distinto dentro de un proceso dado del desarrollo. Éstos se pueden tratar como tipos especiales de clases: tienen atributos y operaciones, pero tienen restricciones especiales en sus relaciones con otros elementos y en su uso.

Un estereotipo se basa en un elemento de modelado existente. El contenido de información del elemento estereotipado es igual que el elemento de modelo existente. Esto permite que una herramienta almacene y manipule el nuevo elemento de la misma forma que lo hace con el elemento existente. El elemento estereotipado puede tener sus propios iconos; es fácil que una herramienta lo permita. Por ejemplo, una “organización de negocios” podría tener un icono que parece un grupo de personas. El estereotipo puede también tener una lista de las restricciones que se aplican a su uso. Por ejemplo, quizás una “organización de negocios” se puede asociar solamente a otra “organización de negocios” y no a cualquier clase. No todas las restricciones se pueden verificar automáticamente por una herramienta de propósito general, pero

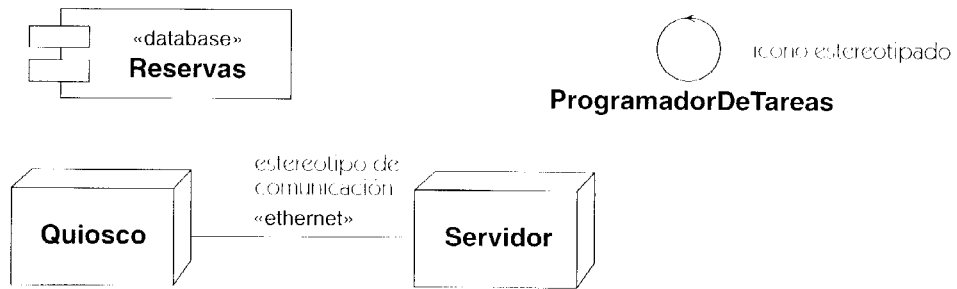


Figura 11.3 Estereotipos

pueden hacerse cumplir manualmente o ser verificadas por una herramienta añadida que entienda el estereotipo.

Los estereotipos pueden utilizar valores etiquetados para almacenar las propiedades adicionales que no se pueden modelar con el elemento base.

Los estereotipos se muestran como cadenas de texto rodeadas por comillas («») puestos dentro o cerca del símbolo normal del elemento. El modelador puede también crear un icono para un estereotipo particular, que sustituye el símbolo normal del elemento (Figura 11.3).

Adaptación de UML

Los mecanismos de extensión de restricciones, de valores etiquetados, y de estereotipos permiten adaptar los perfiles de UML para los dominios especiales de uso. Se han realizado varios perfiles, que se describen en el apéndice C, Extensiones de Proceso. Otros han sido propuestos por los usuarios. La capacidad de adaptación del lenguaje de modelado significa que los dominios de uso pueden adaptar el lenguaje de modelado a sus necesidades, sin embargo, todavía comparte la extensa preponderancia de los conceptos genéricos y comunes a todos los dominios.

Descripción

Los modelos de UML se utilizan dentro de un entorno. La mayoría de la gente utiliza el modelado como medio para un fin —vamos a llamarlo desarrollo de buenos sistemas— y no como fin en sí mismo. El propósito y la interpretación del modelo están afectados por el resto del entorno. Otras facilidades en el entorno más amplio incluyen los metamodelos que abarcan muchos lenguajes, herramientas de edición del modelo, lenguajes de programación, sistemas operativos y componentes importantes del sistema, y el mundo de los negocios y la ingeniería dentro del cual se utilizan los sistemas. La responsabilidad de dar significado a un modelo y de implementar su objetivo se resuelve con todas estas facilidades, incluyendo UML.

Los modelos existen en varios niveles de concreción. UML es un lenguaje de modelado, de propósito general, que incluye semántica y notación, pero es utilizable con diversas herramientas y lenguajes de implementación. Cada nivel de uso introduce ciertas consideraciones de modelado, que aparecen en UML en diferentes grados.

Responsabilidades semánticas

Un metamodelo es la descripción de un modelo. Un lenguaje de modelado describe modelos; por lo tanto, puede ser descrito por un metamodelo. Un metamodelo procura hacer un lenguaje exacto, definiendo su semántica, pero hay una tensión para permitir las extensiones para nuevas situaciones. La forma real del metamodelo es importante para la implementación de las herramientas, y el intercambio de modelos, pero no muy importante para la mayoría de los usuarios. Por lo tanto no la hemos incluido en este libro. Los que estén interesados pueden consultar los documentos originales del estándar [UML-9] disponibles en el CD adjunto.

Un metamodelo y un lenguaje deben cubrir muchos campos y acomodar muchas interpretaciones. Los sistemas existentes tienen modelos de ejecución y de memoria diferentes. Es imposible elegir uno de ellos como la interpretación correcta. De hecho, es probablemente engañoso incluso considerar tal opción. En su lugar, uno puede pensar en las diversas interpretaciones de los modelos de ejecución como puntos de variación semántica. Un punto de variación semántica es un punto de diferencia sobre la semántica detallada de la ejecución, pero que es ortogonal a otros aspectos de un sistema. Por ejemplo, un entorno puede elegir o no ofrecer la clasificación dinámica, la capacidad de un objeto de cambiar su clase en tiempo de ejecución. Hoy, la mayoría de los lenguajes de programación no lo permiten, principalmente por razones de implementación del lenguaje de programación, pero algunos sí. La diferencia es indistinguible en la semántica estática. La opción de la clasificación estática o de la clasificación dinámica se puede identificar

como un punto de variación semántica con dos opciones: clasificación estática o clasificación dinámica. Cuando existen tales opciones, la gente discute a menudo sobre cuál es la interpretación correcta. En vez de esto, observe que esto es una elección y asígnele un nombre para poder utilizarla.

Un metamodelo describe el contenido de un modelo bien formado, mientras que un lenguaje de programación describe un programa bien formado. Solamente un modelo bien formado tiene un significado y una semántica apropiada; no tiene sentido pedir el significado de un modelo mal formado. Sin embargo, la mayoría del tiempo, los modelos bajo desarrollo no están bien formados. Son incompletos y posiblemente inconsistentes. Pero esto es lo que deben contemplar las herramientas de modelado: modelos incompletos, no sólo modelos acabados. El metamodelo de UML describe modelos correctos, bien formados. Un metamodelo separado puede describir posibles fragmentos de un modelo. Dejamos a los fabricantes de la herramienta decidir dónde situar el límite en los fragmentos del modelo admitidos, y qué tipo de semántica dar a los modelos mal formados.

UML incluye algunos mecanismos de extensión incorporados para adaptar su uso en dominios especializados. Los mecanismos incluyen la capacidad de definir estereotipos y valores etiquetados. Estos mecanismos se pueden utilizar para adaptar una variante de UML, definiendo un sistema de estereotipos y de etiquetas, y adoptando las convenciones para su uso a la hora de construir un modelo. Por ejemplo, las variantes desarrolladas podrían centrarse en la semántica de la implementación de varios lenguajes de programación. La adición de extensiones puede ser poderosa, pero conlleva algunos peligros inherentes. Dado que su semántica no se define dentro de UML, UML no puede proveer su significado; la interpretación se delega al modelador. Además, si no se tiene cuidado, algunos significados pueden ser ambiguos o incluso inconsistentes. Las herramientas pueden proporcionar significados y funciones automatizadas para los estereotipos y las etiquetas definidas por las herramientas, pero no para las extensiones definidas por el usuario. Sin importar la ayuda para las extensiones, cualquier extensión tira del usuario lejos del centro común que el estándar del lenguaje proporciona y socava las metas de la capacidad de intercambio de modelos y de comprensibilidad de los modelos. Por supuesto, siempre que usted utilice una biblioteca de clases particular, usted diverge de la capacidad de intercambio perfecta de la nada. Así que, no se preocupe sobre lo abstracto. Utilice las extensiones cuando sean de ayuda, pero evítelas cuando no sean necesarias.

Responsabilidades de notación

La notación no agrega significado a un modelo, sino que ayuda al usuario a entender su significado. La notación no tiene semántica, sino que agrega a menudo las connotaciones para un usuario, tal como la afinidad percibida de dos conceptos basados en su proximidad en un diagrama.

Los documentos de UML [UML-98] y este libro definen una notación canónica de UML, que podría llamarse formato de publicación para modelos. Esto es similar a muchos lenguajes de programación en los cuales los programas dentro de artículos publicados se imprimen en un formato atractivo con disposición cuidadosa, palabras reservadas en negrita, y separan las figuras para cada procedimiento. Los verdaderos compiladores tienen que aceptar una entrada más sucia. Esperamos que esas herramientas de edición extiendan la notación a un formato de pantalla, incluyendo las cosas tales como el uso de tipografías y de color para resaltar elementos; la capacidad de suprimir y de filtrar fácilmente elementos que no son actualmente de interés, para